CHAPTER I

1. INTRODUCTION

The Secure Drive: Intelligent Vehicle Protection and Safety System is an innovative, next-generation IoT-based security solution developed to provide enhanced protection against vehicle theft and unauthorized access. With the increasing number of theft cases and the growing sophistication of intruders, conventional security mechanisms such as basic alarms, steering locks, or mechanical immobilizers are no longer sufficient. Secure Drive addresses these vulnerabilities by integrating intelligent automation, advanced biometric technologies, and AI-driven recognition methods to create a robust, multi-layered security environment. Through a carefully designed architecture built on the Raspberry Pi 4, the system ensures high processing power, scalability, and cost efficiency, making it suitable for both commercial and personal vehicles.

One of the primary challenges faced in vehicle security is the ability to accurately verify the identity of the person attempting to access or start the vehicle. Secure Drive overcomes this by implementing a four-layer authentication mechanism that includes face recognition, fingerprint scanning, voice command authentication, and a secure 4-digit passcode system. Each of these layers adds an additional barrier, ensuring that even if one security measure is compromised, the remaining layers prevent unauthorized operation. AI-powered facial recognition identifies the legitimate driver with high accuracy, while the fingerprint scanner provides a physical biometric verification. Voice command authentication adds another secure interaction method that cannot be easily replicated by intruders. Finally, the passcode ensures an additional manual layer of control that the owner can rely on in case of emergency.

In addition to identity verification, Secure Drive extends its functionality to real-time monitoring and active protection. The system integrates GPS tracking, which allows continuous monitoring of the vehicle's location. This proves invaluable in the event of theft or unauthorized movement, always providing precise coordinates. The GSM module enhances this capability by sending instant SMS alerts to the owner whenever suspicious activities are detected—such as failed authentication attempts, forced entry, or tampering with vehicle systems. Furthermore, the system features an automatic intruder capture mechanism, where the camera instantly records and transmits an image of

the unauthorized individual, enabling faster identification and supporting law enforcement during vehicle recovery.

The layered nature of Secure Drive's architecture creates not only a security system but also a smart monitoring environment. With the growing emphasis on intelligent transportation and smart vehicle ecosystems, Secure Drive contributes to safer driving experiences through optional modules such as drowsiness detection, obstacle sensing, and automatic speed regulation. These safety features ensure the driver remains attentive and alert, helping to prevent accidents caused by fatigue or sudden obstacles. The system's embedded sensors and automation capabilities allow real-time decision-making that supports both security and safety, forming an integrated solution that goes beyond traditional theft prevention.

From a technological perspective, Secure Drive exemplifies the seamless fusion of embedded systems, artificial intelligence, and IoT communication. By leveraging the processing capability of Raspberry Pi, the accuracy of biometric modules, the flexibility of Python programming, and the connectivity of GPS/GSM networks, the system delivers a reliable and sophisticated vehicle protection framework. The modular design ensures that components can be upgraded or expanded as needed, making Secure Drive adaptable to future advancements in smart vehicle technology. Its IoT-based communication infrastructure enables real-time data sharing and remote monitoring, always offering vehicle owners greater control and awareness of their vehicle's status.

Overall, the Secure Drive system redefines the concept of vehicle safety by providing comprehensive protection through multi-factor authentication, intelligent monitoring, and real-time communication. Its combination of advanced sensors, AI-powered verification, and IoT integration creates an automated security ecosystem that minimizes the risk of theft while enhancing vehicle safety. By building on modern technological innovations and providing multiple layers of defense, Secure Drive stands as a reliable, efficient, and futuristic solution to the growing challenges in vehicle security. It strengthens user confidence, improves safety standards, and represents a significant step toward the development of truly smart and secure transportation systems in the modern world

CHAPTER II

2. LITERATURE REVIEW

[1] This study proposes a Raspberry Pi–based multi-factor vehicle security system using face and fingerprint authentication. Biometric verification is required before vehicle ignition. Data preprocessing improves accuracy under different conditions. Experimental results show high reliability and low false acceptance. The work highlights Raspberry Pi scalability for IoT security applications.

[2] The authors present an IoT-enabled face recognition vehicle anti-theft system using Raspberry Pi 4. Facial authentication is performed before engine access is allowed. Image preprocessing improves robustness against lighting variations. Real-time alerts notify the vehicle owner of intrusions. Results confirm fast and reliable embedded performance.

[3] This paper introduces a gesture-learning robot control system using machine-learning techniques. Hand gestures are translated into motion commands. The system improves human–robot interaction without complex hardware controllers. Experimental results show accurate gesture recognition. Applications include automation and robotics education.

[4] The study proposes an intelligent driver assistance system integrating obstacle detection, accident alerts, and fatigue monitoring. Embedded sensors enable real-time analysis to reduce human error. Test results show timely warnings and reliable detection. The system is cost-effective and scalable. The research supports safer driving environments.

[5] This foundational research presents a GPS–GSM vehicle tracking system for theft prevention. GPS provides location data while GSM transmits alerts. The system demonstrates reliable tracking and fast message delivery. It is suitable for fleet management and emergency services. Low cost supports wide adoption.

[6] This paper presents an underwater navigation system using metal detectors for object detection. Embedded signal processing improves detection accuracy. Noise and signal attenuation are addressed using filtering techniques. Experimental results confirm feasibility. Applications include marine inspection and defense.

[7] The authors develop a biometric vehicle security system using fingerprint authentication with GPS and GSM. Unauthorized access triggers tracking and SMS alerts. Experimental results show quick recognition and stable communication. The system balances security and convenience.

[8] This study proposes an IoT-based intelligent monitoring and control system for smart appliances. Sensors collect environmental data and control devices automatically. Wireless communication enables remote monitoring. Experimental results show reliable data transmission. The system is scalable across multiple domains.

[9] This research presents an embedded anti-theft vehicle control system with engine immobilization. Sensors detect unauthorized access and trigger alerts. Testing confirms fast response and stable operation. The system emphasizes power efficiency and fault tolerance. It forms a base for IoT security models.

[10] This paper introduces an IoT-based embedded system for vehicle security and driver monitoring. GPS and GSM provide real-time tracking and alerts. The system enhances safety through continuous monitoring. Experimental results show accurate performance. The work highlights scalability for advanced analytics.

[11] This study presents a CAN-based vehicle theft prevention system for secure electronic locking and unlocking. The use of CAN ensures reliable, fast, and interference-resistant communication. Microcontrollers and CAN transceivers manage vehicle access control. Experimental results show stable operation and quick response

[12] The authors propose an IoT-driven vehicle security framework using layered sensing and communication. The system enables real-time monitoring, intrusion detection, and mobile alerts. Experimental validation confirms reliable data transmission. Challenges such as latency and power consumption are addressed. The architecture is scalable for biometrics and remote immobilization.

[13] This paper introduces a GPS–GSM-based vehicle security and tracking system. Unauthorized access triggers real-time alerts and continuous location tracking. Experimental results demonstrate accurate positioning and reliable communication. The system supports remote monitoring by vehicle owners. Its low cost makes it suitable for large-scale deployment.

[14] The study presents a smart helmet system to improve rider safety by preventing ignition under unsafe conditions. Sensors detect alcohol usage, drowsiness, and improper helmet wear. Microcontroller processing ensures fast decision-making. Testing shows reliable sensor operation and communication. The work highlights wearable safety enforcement for accident prevention.

[15] This early research proposes an autonomous vehicle monitoring and tracking framework using GPS and communication modules. The system provides real-time location updates and emergency alerts. Experimental results confirm dependable tracking accuracy. The architecture is adaptable across vehicle types. Applications include fleet management and public transport security.

[16] This paper discusses advanced security systems using detection, imaging, and biometric anomaly recognition. Microwave imaging and signal processing improve threat detection accuracy. Biometric anomaly analysis identifies unusual patterns. Experimental studies show feasibility in real environments. The work highlights multi-sensor fusion for enhanced security.

[17] The authors develop an embedded anti-theft vehicle control system using sensors and microcontrollers. Unauthorized access triggers immobilization and alert mechanisms. The system provides fast response and stable performance. Experimental validation confirms effectiveness under different conditions. The framework remains relevant for modern vehicle security design.

[18] This study proposes an IoT-based embedded vehicle security and driver monitoring system. Sensors and communication modules track vehicle status and abnormal behavior. GPS and GSM enable real-time alerts. Testing shows reliable data transmission. The system is scalable for advanced analytics and automation.

[19] The research presents a CAN-based electronic vehicle lock and unlock mechanism. CAN communication provides fast execution and EMI resistance. The architecture integrates easily with automotive subsystems. Testing confirms reliable lock control. The approach reduces vulnerability compared to mechanical locks.

[20] This paper explores an IoT-enabled vehicle security architecture with cloud-based monitoring. The system supports intrusion detection, tracking, and remote alerts. Experimental results show accurate detection and seamless communication. Security and data integrity challenges are addressed. The framework supports biometric and remote ignition extensions.

CHAPTER III

3. PROBLEM STATEMENT

Vehicle theft and unauthorized access remain major global security challenges, causing significant financial losses and personal safety risks, especially as modern vehicles become more technologically advanced while criminal methods continue to evolve. Traditional key-based and remote locking systems are increasingly vulnerable to duplication, hacking, and relay attacks, allowing intruders to unlock and start vehicles without physical keys. Existing alarm-based systems provide minimal protection because they lack intelligent monitoring, real-time intrusion detection, and instant communication, meaning owners often become aware of theft only after the vehicle has already been taken. Furthermore, most current solutions do not incorporate robust multi-factor authentication, relying instead on single-layer mechanisms that are easier to bypass. Although technologies such as face recognition, fingerprint scanning, voice authentication, and passcode verification are individually available, they are rarely integrated into a unified security framework; combining these methods into a layered authentication system greatly enhances protection, ensuring that even if one security layer is compromised, additional layers remain active to prevent unauthorized vehicle access.

CHAPTER IV

4. OBJECTIVES

The primary objective of this project is to design and implement Secure Drive: Intelligent Vehicle Protection and Safety System, which leverages IoT and biometric technologies to enhance vehicle security and driver safety. The specific objectives are as follows:

- The system uses multi-level IoT-based authentication to solve the problem of traditional locks being easily hacked or bypassed.
- Face and fingerprint recognition directly address the issue of unauthorized access by providing strong biometric verification instead of relying on vulnerable physical keys.
- Voice command integration improves convenience while ensuring secure user interaction, solving the problem of outdated and rigid vehicle control methods.
- The emergency override mechanism provides a controlled fallback path to enable ignition when primary biometrics (face and fingerprint) cannot be completed due to sensor unavailability, environmental conditions, or input failure.
- Real-time GSM alerts, GPS tracking, and image capture overcome the limitations of ordinary alarms by giving the owner immediate information about intrusion attempts.
- A fully integrated embedded hardware–software system using Raspberry Pi, camera, fingerprint sensor, GSM, and GPS meets the need for a unified, intelligent, and reliable vehicle protection solution against modern theft techniques.

CHAPTER V

5. MATERIALS AND METHODOLOGY

5.1 Hardware requirements

5.1.1 Raspberry pi 4

The system design chooses a central processing board that can run authentication algorithms, sensor polling, serial communication, and ignition control reliably while consuming minimal power. The core board category fits the class of energy-efficient single board computers like the Raspberry Pi 4, which is selected because it supports on-device AI processing for face matching, optical UART communication for both GPS and GSM modules, sufficient GPIO pins for relay and keypad interfacing, and multi-threaded Python execution without requiring external cloud hardware. It delivers strong performance per watt using optimized ARM-based computation.

Interface planning around the core board considers automotive electrical noise, voltage safety, and runtime stability. Since most external modules operate at 5V logic while the board GPIO works at 3.3V, level shifters are included to protect pins from over-voltage stress. The board also allows selective activation of power-hungry units (like camera and GSM radio) only when needed, avoiding continuous streaming or unnecessary polling that can degrade a vehicle battery.

Fig 5.1 illustrates the Raspberry Pi 4 Model B, highlighting its major hardware components such as the Broadcom processor, USB ports, Ethernet interface, HDMI ports, and GPIO header

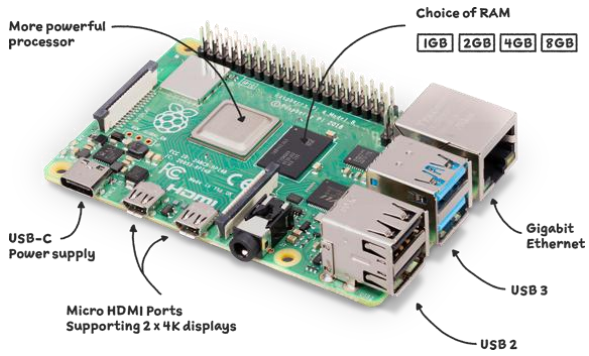


Fig 5.1 Raspberry pi 4

5.1.2 Raspberry pi 5MP Camera Module

The imaging subsystem plays a dual and critical role in the Secure Drive architecture—primary driver identity verification and intrusion evidence acquisition—both executed locally on the automotive embedded board. A CSI (Camera Serial Interface) camera is selected over USB cameras due to its tighter hardware integration, deterministic streaming path, low-latency frame capture, and reduced CPU overhead, which is essential for maintaining real-time authentication performance without stalling the ignition pipeline. The camera delivers continuous facial frames only during active authentication windows, where the perception model detects, aligns, and crops the face region with a lightweight CNN detector, followed by embedding generation and profile comparison through an on-device AI verification model. This ordered visual pipeline ensures high accuracy by rejecting multi-face interference, background noise, or false object contours, confirming that the person attempting ignition is the registered authorized driver. for intrusion monitoring, the camera is intentionally triggered only on suspicious or repeated biometric verification failures, operating in an event-driven capture mode to conserve power and processing. The moment an unauthorized access attempt is detected, the camera takes a still image snapshot, stores it to local non-volatile memory, optionally uploads it later to authenticated cloud storage once the Google account token handshake is confirmed, and tags the incident with system timestamp and location data retrieved from dedicated GPS UART handlers.

Fig 5.2 depicts the Raspberry Pi 5MP Camera Module, a compact imaging device designed for seamless integration with Raspberry Pi boards via the CSI interface. It is capable of capturing high-quality images and videos, making it suitable for applications such as real-time monitoring, face detection, computer vision, and security-based embedded systems.



Fig 5.2 Raspberry Pi 5MP Camera

5.1.3 R307S fingerprint module

The biometric subsystem in Secure Drive relies on an optical fingerprint reader to establish driver identity through enrolled-template mapping rather than raw image comparison. Optical fingerprint sensors are preferred in automotive embedded environments because they deliver stable ridge–valley scanning, operate reliably under moderate pressure variance, and provide digital biometric templates that can be stored and matched on-device with calibrated acceptance thresholds. During initial system setup, each authorized driver's fingerprint is enrolled multiple times to generate high-confidence templates, ensuring the stored signature absorbs minor inconsistencies caused by finger placement angle, motion blur, or surface dust. At runtime, the sensor captures a fingerprint sample and compares it against the enrolled database, confirming that the person verified in the preceding face recognition stage is physically present, adding strong resistance against photo or video spoof attempts aimed at bypassing ignition security.

To maintain system reliability, the biometric unit performs internal scan-quality checks that validate signal clarity before matching begins, and confidence scoring is tuned to minimize false acceptance while avoiding unnecessary false rejections

Fig 5.3 illustrates the R307S Fingerprint Module, a biometric authentication device used for reliable user identification in embedded security systems. It captures and processes fingerprint patterns internally and communicates with the controller via serial interface, enabling fast and accurate access control and authentication applications.



Fig 5.3 R307S Fingerprint Module

5.1.4 USB Microphone

The voice command unit is the third stage of your ignition authorization system and is designed to verify driver intent, not identity. You used a USB microphone connected to the core embedded controller and handled speech decoding entirely on-device using an offline speech-recognition library. Unlike cloud-based voice assistants, this module processes audio locally to reduce network dependency, latency, and power overhead, making it reliable even in remote areas or low-connectivity conditions. After face and fingerprint validation succeed, the system enables the voice listener thread, which listens only for a consciously spoken ignition phrase such as *"Start"*. The command is analyzed into a confidence score before execution, ensuring that background noise, accidental speech, or unrelated audio cannot trigger ignition.

Once the system confirms a valid and high-confidence command, the ignition actuation layer is unlocked by energizing the 5V relay that physically gates the vehicle motor/engine power. If speech confidence is low, the system requests a retry without freezing other monitoring threads. This module can be described in your report as a command-based ignition authorization layer that follows secure activation rules, operates offline, avoids false triggers, and integrates non-blocking multithreaded listening to maintain real-time performance .

Fig 5.4 shows the USB Microphone, which is used for capturing voice input in embedded and IoT-based systems. It enables voice command recognition and audio-based interaction by providing clear digital audio signals directly to the controller through the USB interface.



Fig 5.4 USB Microphone

5.1.5 NEO 6M GPS and SIM900A GSM Module

The telematics layer in your project is the system's remote awareness and tracking backbone, built using tightly scheduled serial communication modules that deliver only essential security information. The positioning block relies on a GNSS receiver to extract latitude and longitude from live NMEA sentences, enabling precise vehicle localization even without IP connectivity. In parallel, the cellular communication unit sends SMS alerts directly to the owner's phone when an intrusion or emergency override event occurs. These two modules are isolated on independent UART interfaces to avoid serial bus contention, and GPS streaming is disabled or buffered after retrieving a single clean coordinate sentence per event, preventing buffer overflow, high power bursts, or ignition stalling during authentication. The SMS payload can include structured fields such as incident type, timestamp, and coordinate data, giving actionable owner visibility without relying on cloud processing.

The GSM alerting subsystem performs AT-command based SMS dispatch with controlled retry governance and cooldown gaps, which helps maintain link reliability under automotive electrical noise

Fig 5.5 illustrates the NEO-6M GPS module and SIM900A GSM module, which together enable location tracking and wireless communication in the system. The GPS module provides real-time latitude and longitude coordinates, while the GSM module facilitates SMS alerts and data transmission for remote monitoring and security notifications.



Fig 5.5 NEO6M(GPS) and SIM900A(GSM)

5.1.6 Relay and LCD Display Module

The actuation and feedback layer relies on three supportive hardware blocks—ignition relay control, motor power gating, and visual status output through an LCD display. The ignition lock is implemented using a 5V relay module driven by GPIO logic from the embedded controller, which works as a physical gatekeeper between the vehicle battery rail and the motor/engine circuit. The relay remains OFF in locked and monitoring states, ensuring zero-power leakage to the ignition line. After the system completes the primary authentication chain—face verification, fingerprint matching, and a validated voice ignition command—the controller switches the relay to ON, energizing the motor power line and allowing the vehicle to start. The motor, represented by the vehicle's drive load, is controlled indirectly through this relay, making the security hardware-independent from the vehicle model and adaptable for real automotive ignition circuits.

User and owner awareness is strengthened using a display feedback unit, the LCD interface, connected through GPIO/I2C with 3.3V logic safety. Instead of constantly printing logs, the LCD shows system states such as Face matched or unmatched, Fingerprint matched, Voice, motor ON, or passkey matched or unmatched Override Active, making the system interaction clear and evaluator friendly .

Fig 5.6 shows the Relay module and 16×2 LCD display, which are used for actuation and system status indication respectively. The relay provides electrical isolation and controls high-power loads such as the ignition circuit, while the LCD displays real-time system messages, authentication status, and operational alerts.

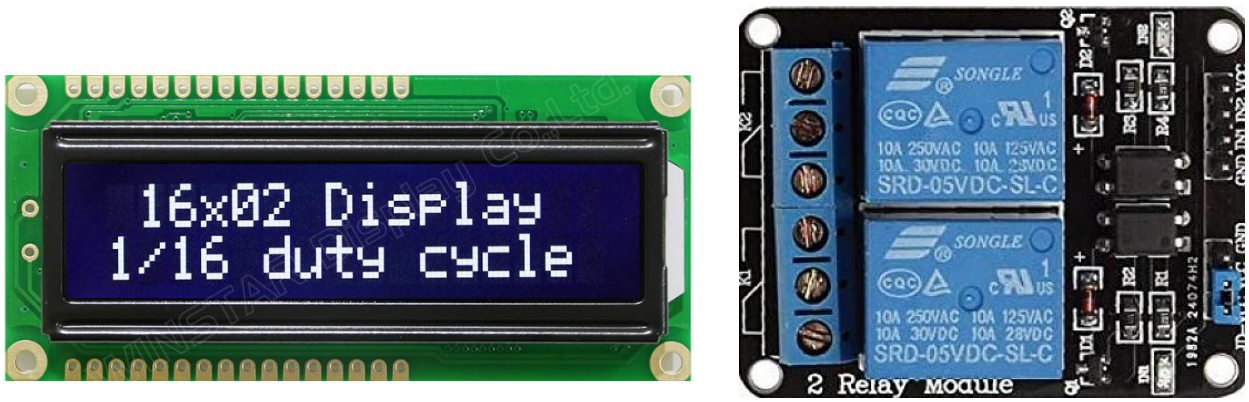


Fig 5.6 Relay and LCD(16*2) module

5.2 Software requirements

5.2.1 Operating System Requirements

The system operates on Raspberry Pi OS, a Linux-based operating system optimized for embedded and IoT applications. It provides a stable platform with built-in support for essential interfaces such as GPIO, I2C, SPI, and UART, enabling seamless communication with the fingerprint sensor, GPS module, GSM modem, relay driver, and LCD display. The OS includes native drivers for the CSI camera interface, ensuring smooth image capture for face recognition and intrusion evidence. It supports USB audio devices, allowing the microphone to function reliably for voice-command detection. Raspberry Pi OS also offers a powerful Python runtime environment, enabling the installation and execution of AI, biometric, and cloud-communication libraries such as OpenCV, MTCNN, dlib, pySerial, SpeechRecognition, and Google API client tools. Its lightweight design ensures low system overhead and fast boot times, which are essential for real-time security applications. Additionally, regular firmware updates enhance system stability, camera performance, and UART reliability. The OS provides robust networking capabilities, including Wi-Fi and Ethernet, which support cloud services during evidence uploads. Built-in security features and Linux file permissions help protect stored biometric data and local system logs. Overall, Raspberry Pi OS enables a reliable, secure, and fully compatible software foundation for integrating all modules in the vehicle protection system.

5.2.2 Python Environment Setup

The system relies on Python 3.x as the primary programming language due to its vast library support and compatibility with Raspberry Pi OS. Python provides high-level abstractions for handling GPIO operations, camera interfaces, and serial communications. It offers powerful scientific and AI modules needed for face detection, feature extraction, and biometric decision-making. The interpreter supports real-time execution of multithreaded processes, enabling simultaneous handling of camera frames, GSM communication, and keypad monitoring. Package managers like pip are essential for installing project dependencies such as OpenCV, NumPy, pySerial, and SpeechRecognition. Python's extensibility allows seamless integration of external APIs including Google Drive for evidence uploads. Error logging, exception handling, and modular code structuring are supported natively, improving reliability. Python's strong community and

documentation ensure fast debugging and future scalability. Additionally, its compatibility with both hardware-level libraries and cloud services makes it an ideal choice for hybrid IoT-AI systems. Overall, Python provides the computational backbone for all security, recognition, and communication tasks in the project.

5.2.3 OpenCV Vision Tools

OpenCV is required for real-time image acquisition, preprocessing, and facial feature analysis. It enables efficient conversion of camera YUV frames into RGB/BGR formats necessary for face recognition algorithms. The library provides functions for cropping, resizing, histogram equalization, and other enhancements that improve face detection accuracy. OpenCV works seamlessly with the Raspberry Pi camera module, ensuring minimal latency during live capture. It supports integration with neural network models used for embedding generation and recognition. Drawing overlays, bounding boxes, and confidence levels on frames is made possible using OpenCV visualization tools. It also allows saving unknown faces for evidence documentation. The optimized C++ backend delivers fast processing, even on embedded platforms. OpenCV's Python bindings make it easy to merge AI logic with the rest of the system. OpenCV is essential for building a reliable, accurate, and efficient facial authentication workflow.

5.2.4 Biometric Model Support

The system requires pretrained neural models for face recognition and fingerprint validation. Tools like face_recognition (dlib) generate high-quality 128-dimensional embeddings that allow precise user identification. These models perform convolutional feature extraction, enabling recognition even under variations in lighting, angle, or facial expressions. Fingerprint verification is handled through algorithms in the PyFingerprint library, supporting real-time template matching. Both engines must operate efficiently on limited embedded hardware, so lightweight, optimized models are preferred. Machine learning inference runs offline, improving security and reducing dependency on cloud computing. Embedding comparison and thresholding logic ensures accurate differentiation between authorized and unknown users. The biometric layer serves as the core of multi-level authentication, making reliable model performance essential.

5.2.5 UART Communication Drivers

The project relies heavily on serial communication, requiring stable UART drivers for GSM, GPS, and fingerprint modules. Python's pySerial library enables reading and writing of AT commands, NMEA sentences, and fingerprint data packets. The OS must support multiple UART channels simultaneously without conflicts, ensuring dedicated ports for GSM, GPS, and the fingerprint sensor. Buffer-flush routines are required to remove stale data packets and prevent parsing errors. Timeouts, baud-rate configurations, and packet-validation rules ensure reliable communication even during automotive noise interference. UART error handling prevents crashes when modules disconnect or send corrupted data. These drivers enable transmission of SMS alerts, GPS coordinate acquisition, and fingerprint matching, making them critical to overall system performance.

5.2.6 Security & Encryption

Security modules are essential for protecting sensitive biometric and location data. OAuth 2.0 authentication ensures secure communication with Google Drive during image uploads. Local fingerprint templates must be stored in encrypted form to prevent unauthorized extraction. System logs containing GPS data or authentication attempts require controlled access. Secure hashing methods protect stored credentials and emergency passcodes. The OS must enforce Linux file permission rules to restrict user-level access. HTTPS-based network requests must be mandatory for all API calls. Proper token management prevents misuse of Google API credentials. Overall, robust security handling ensures the system cannot be maliciously manipulated or bypassed.

5.2.7 Cloud API Integration

Google Drive API is required for uploading intruder images and generating shareable evidence links. OAuth authentication flow ensures secure token exchange between the device and Google's servers. The API must handle resumable uploads for stability during poor network conditions. REST-based calls are used to create file metadata, upload media content, and generate publicly accessible URLs. The cloud system must support structured folder management to store evidence chronologically. Error handling ensures that failed uploads are retried automatically. The API's flexibility allows future integration with dashboards or remote monitoring platforms. Cloud connectivity expands the security system's capability beyond local hardware and supports real-time owner notification.

5.3 Proposed Model or Methodology

5.3.1 Overall System Design

The system is built around a Raspberry Pi controller, which manages sensors, biometric checks, communication modules, and ignition control. The camera captures live images of the driver, and these images are processed by an AI model to detect and recognize faces. If the face is verified, the system moves to the next biometric check—the fingerprint sensor—which compares the scanned fingerprint with stored templates. After confirming both biometrics, the system activates a voice listener that waits for a specific ignition command. Only when identity, physical presence, and intention are confirmed does the relay turn ON, enabling vehicle ignition. The GSM, GPS, and camera modules run in parallel threads to capture evidence, extract live location, and send alerts whenever an unauthorized access attempt occurs.

5.3.2 Stepwise Authentication Flow

The authentication starts with face recognition using a CSI camera and AI detection models that isolate and verify the driver's face. If matched, the system triggers the fingerprint stage, where the R307S sensor checks whether the scanned fingerprint matches an enrolled template. Only after both biometrics succeed does the system activate the voice command stage, where the microphone listens for the predefined command such as "Start Vehicle." This voice stage ensures the driver is intentionally starting the vehicle. If the voice command is valid, the relay is enabled, and ignition power is supplied. If face or fingerprint verification fails, the system immediately captures an image, reads live GPS coordinates, and sends an alert to the owner.

5.3.3 Emergency Override Path

The emergency keypad acts as a backup access method and is kept separate from the biometric authentication sequence. It becomes active only when the user intentionally triggers it through a dedicated key input. The Raspberry Pi reads this keypad input on a separate thread so it does not interrupt other processes. When the correct emergency code is entered, the relay is activated for a short, controlled duration. At the same time, the system captures the driver's photo, collects GPS coordinates, and sends an SMS alert to the owner. This prevents misuse and ensures emergency access is always monitored.

5.3.4 IoT Alert Message Structure

The alert sent to the owner contains three essential components:

1. GPS Location: The NEO-6M module provides live latitude and longitude, which are included in the SMS along with a clickable Google Maps link.
2. Captured Image: The camera captures a photo of the intruder, which is saved locally and may be uploaded to Google Drive for secure viewing.
3. SMS Alert: The SIM900A GSM module sends a structured message indicating the incident type, timestamp, location, and link to the stored image. This ensures the owner receives clear, quick, and useful information to respond to a security threat.

5.3.5 System State Transitions

The system follows a clear sequence of states:

- LOCK State: System is idle; ignition is off; sensors are ready.
- AUTHENTICATION State: Performs face → fingerprint → voice checks.
- AUTHORIZED State: Ignition relay activates after successful authentication.
- MONITOR State: Triggered when any authentication fails; captures photo, reads GPS, sends alert.
- OVERRIDE State: Activated only by keypad for emergency access; ignition enabled temporarily with alert.
- RESET State: Clears temporary data, disables relay, resets UART buffers, and returns to LOCK state.

This ensures predictable, secure behaviour and prevents unauthorized activation.

5.3.6 Serial Communication Planning

Each module is assigned its own UART port to avoid data collision:

- GPS uses one UART channel,
- GSM uses another,
- Fingerprint sensor uses another.

Fig 5.7 presents the overall block diagram of the proposed intelligent ignition authorization system, where multiple user authentication inputs such as face recognition, fingerprint verification, voice command, and emergency keypad are processed by Raspberry Pi 4. The decision and control layer validates authorized access before enabling ignition, while the IGF module integrates GPS for real-time location tracking and GSM for sending intrusion alerts with location details to the vehicle owner, ensuring enhanced security and monitoring.

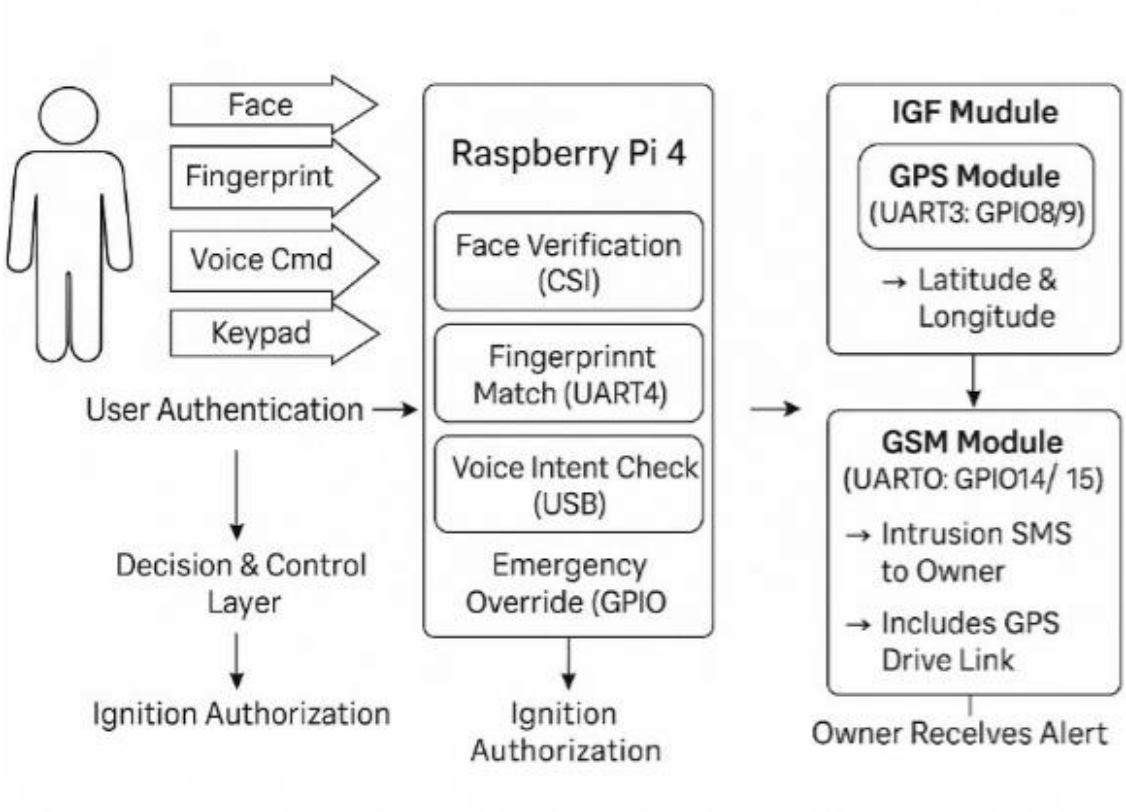


Fig 5.7 Block Diagram

5.4 System schematic and flow diagram

5.4.1 Schematic representation of the system

The complete hardware integration of the system is built around the Raspberry Pi 4, which functions as the central controller responsible for managing sensors, communication modules, alerts, and ignition control. Each peripheral is connected through dedicated and isolated interfaces to ensure stable communication and avoid electrical conflicts. The camera module, used for face recognition and intrusion evidence, is connected directly to the CSI/CSA camera port. This dedicated high-speed interface allows real-time image capture without using USB or GPIO bandwidth, enabling fast facial authentication and immediate photo capture whenever an unauthorized attempt is detected.

The fingerprint sensor (R307S) forms the second biometric layer and is interfaced through UART4, with its TX and RX lines connected to GPIO 16 and GPIO 17. A regulated 3.3V or 5V power line and a common ground ensure stable operation and reliable template matching. In parallel, the telematics subsystem consists of two independent serial modules: the GSM module (SIM900A) connected to UART0 through GPIO 14 and 15 for SMS alerts, and the GPS module (NEO-6M) connected to UART3 via GPIO 9 and 8 for continuous location tracking. Using separate UART ports prevents their data streams from overlapping,allowing GPS coordinates and SMS communication to operate smoothly during intrusion events.

Ignition control is managed through a 5V relay module, where the vehicle battery's positive terminal is wired to the relay COM pin and the ignition line to the NO pin. The relay is triggered by the Raspberry Pi through GPIO 16, but because the same pin is shared with the fingerprint module, a transistor driver or opto-isolator is recommended to prevent GPIO overload. The relay only energizes after successful face, fingerprint, and voice authentication. System feedback and alerts are provided by a buzzer connected to GPIO 7, receiving power through the 3.3V or 5V rail, and used to indicate matches, mismatches, and override events.

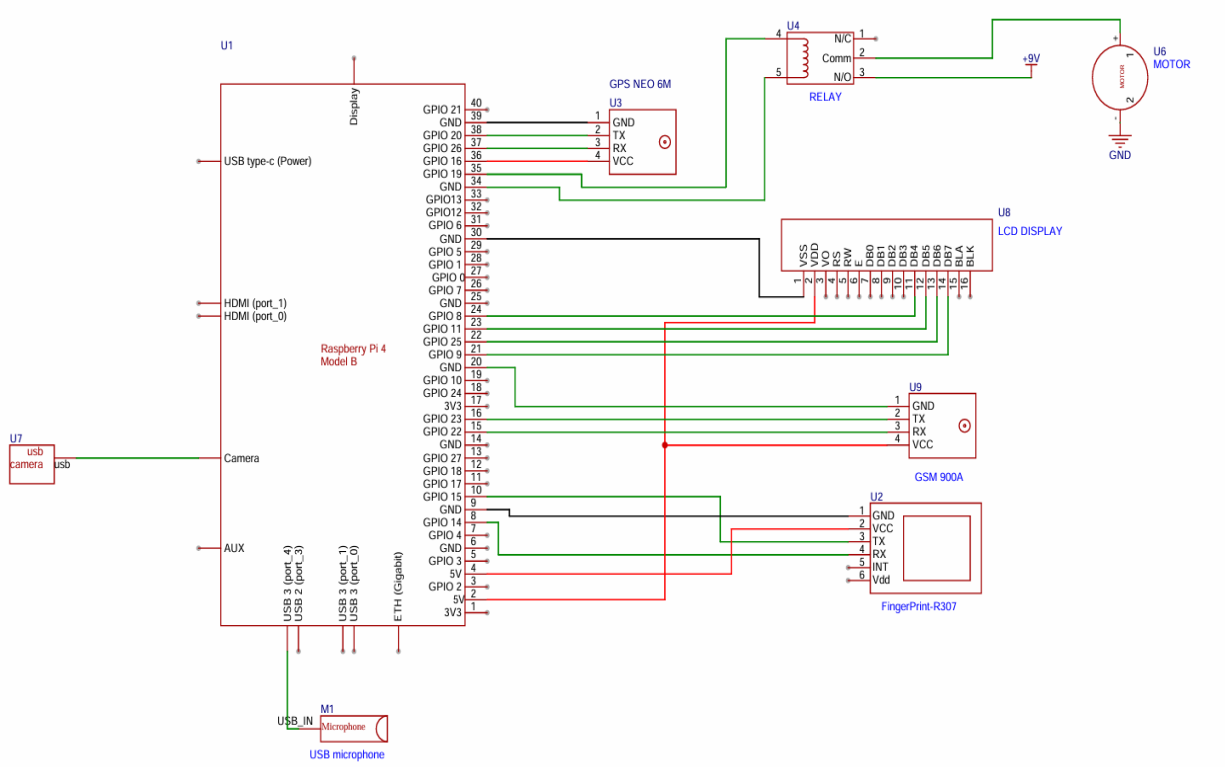


Fig 5.8 Circuit or Schematic Connection

Fig 5.8 shows the schematic (circuit) connections of the proposed system, where the Raspberry Pi 4 Model B acts as the central controller interfacing with the fingerprint module, GPS (NEO-6M), GSM (SIM900A), USB camera, USB microphone, relay module, and 16×2 LCD display through GPIO, UART, USB, and CSI interfaces. The relay is used to control the motor/ignition circuit, while the GSM and GPS modules enable alert messaging with real-time location, ensuring secure ignition authorization and effective intrusion monitoring.

An I2C-based LCD display, connected through GPIO 2 (SDA) and GPIO 3 (SCL) with 5V and ground, provides real-time status messages such as "Face Matched," "Fingerprint OK," "Command Accepted," "Ignition Enabled," or "Intrusion Detected." The USB microphone requires no GPIO pins and plugs directly into a USB port, providing audio input for voice-command verification. All components share a regulated 5V/3.3V power supply from the vehicle's battery through a DC-DC converter, with a unified ground reference maintained across all modules. Proper noise filtering, grounding discipline, shielding of serial lines, and isolation of the relay coil ensure stable and safe operation in noisy automotive electrical environments.

5.4.2 System flowchart representation

The flowchart illustrates the complete working process of the multi-layer vehicle security system, beginning with the system start or boot stage, where all sensors and modules are initialized. Once active, the system immediately performs face authentication using the camera. If the detected face matches a registered user, the process moves to fingerprint authentication; if it fails, the system triggers a security event. During a security event, the system captures a real-time photo of the intruder, collects GPS coordinates, and sends an SMS alert to the owner. After this, it checks whether the failed attempts indicate repeated intrusion. If not, the system returns to the authentication flow; if repeated attempts are detected, it enters lockdown mode with the alarm activated to prevent further unauthorized activity.

During lockdown, normal authentication is blocked, and the system evaluates whether the emergency override option is being used by the legitimate owner. If the owner requests an override, they must enter a passcode through the keypad. A correct passcode allows temporary ignition through a timer-based relay activation, while an incorrect passcode sends an intrusion alert message to the owner. Meanwhile, if the face authentication had succeeded earlier, the system proceeds to fingerprint verification; a successful fingerprint match leads to voice authentication, while a failure again triggers the security event procedure.

Voice authentication acts as the final confirmation of user intent. If the predefined voice command is recognized correctly, if the face authentication had succeeded earlier, the system proceeds to fingerprint verification; a successful fingerprint match leads to voice authentication, while a failure again triggers the security event procedure. During a security event, the system captures a real-time photo of the intruder, collects GPS coordinates, and sends an SMS alert to the owner. After this, it checks whether the failed attempts indicate repeated intrusion. If not, the system returns to the authentication flow; if repeated attempts are detected, it enters lockdown mode with the alarm activated to prevent further unauthorized activity. the ignition relay is activated, enabling the vehicle to start. If the voice command fails, the system again enters the security event stage. Once ignition is enabled, the process terminates cleanly at the stop state. This structured and layered flow ensures that the vehicle can only be started when all required biometric and intent checks are satisfied, while also providing strong intrusion detection, lockdown mechanisms, and emergency access pathways for real-world reliability.

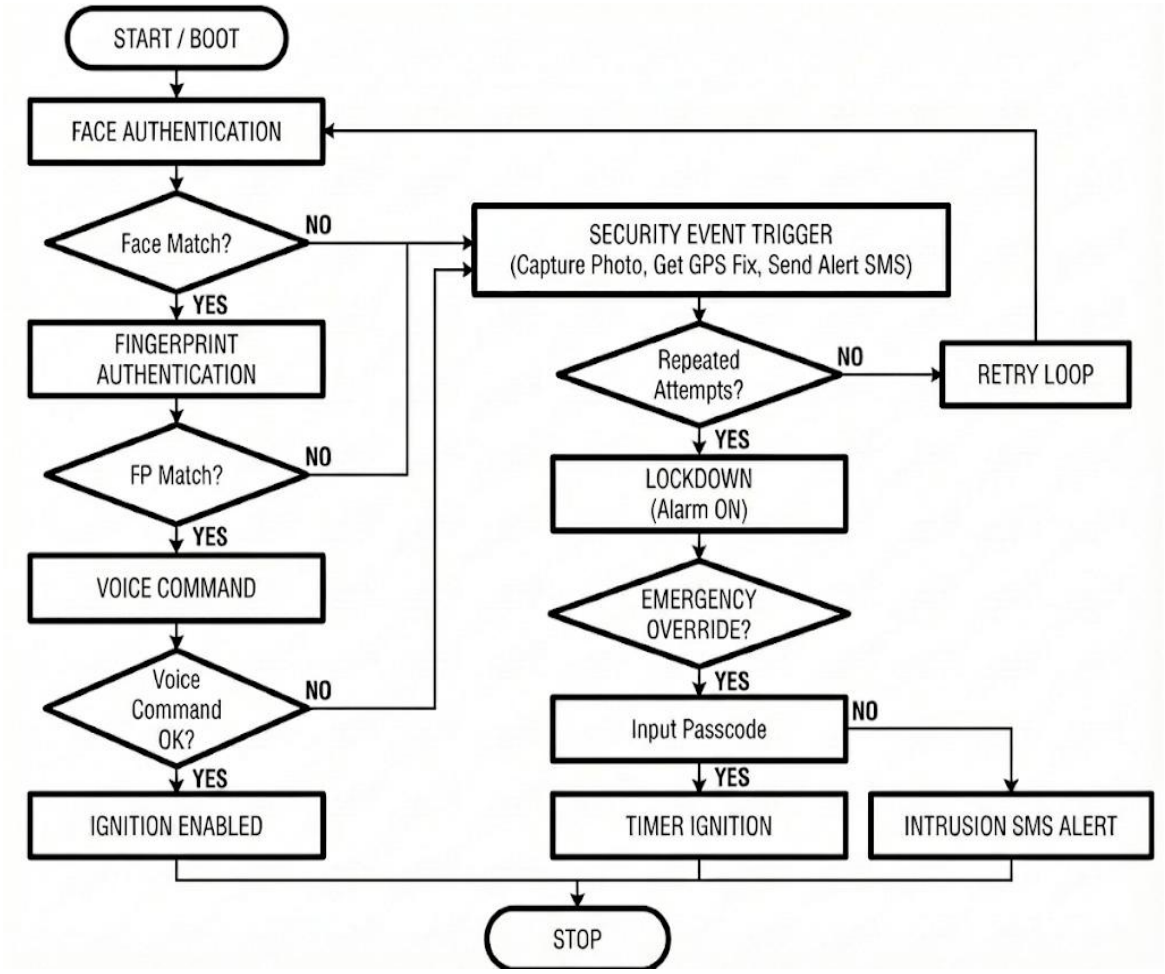


Fig 5.9 Flowchart for the system

Fig 5.9 illustrates the flowchart of the proposed system, detailing the sequential authentication process starting from face recognition, followed by fingerprint verification and voice command validation before enabling ignition. In case of authentication failure, security events are triggered with GPS tracking and SMS alerts, while repeated attempts lead to system lockdown; an emergency override with passcode and timer-based ignition is provided as a controlled fallback mechanism.

5.5 System Workflow

5.5.1 Face Authentication

Face-perception AI in this project is enabled using a combination of optimized computer-vision, biometric encoding, numerical similarity comparison, neural face localization, and IoT-ready file handling libraries. The camera frames are processed using OpenCV, which captures images, converts color spaces, resizes frames, draws UI overlays, and crops detected face regions. To detect and localize faces accurately before generating embeddings, the model uses the CNN-based face detector MTCNN, which scans the frame at different scales, suppresses background noise, ignores multi-face interference, and returns bounding-box coordinates of the detected face. Once the face location is confirmed, biometric feature extraction is done through face_recognition, which internally uses the dlib neural model to convert face pixels into a 128-dimension face embedding vector, a mathematical representation of unique facial structures rather than a stored photograph. These embeddings are compared using compare_faces() (checks if embedding matches DB entries within tolerance) and face_distance() (calculates numerical similarity gap), forming the foundation of secure identity verification.

For storing and retrieving authorized face embeddings after initial enrollment, the system serializes data using the Python object storage utility pickle, saving computed embedding vectors and mapped user labels into reusable encoding files (encodings.pkl), eliminating the need to recompute face models at every boot and reducing CPU, memory, and power cycles. Owner alerts and traceability are handled separately using the UART modem interface SIM900A which sends SMS packets, and GNSS positioning parser NEO-6M which extracts latitude-longitude from live satellite NMEA sentences. Web API communication (used only in intrusion mode, not for identity login) is executed using the light HTTP request handler requests, enabling reverse geocoding and Drive API operations to run in short, scheduled bursts.

To enable Drive photo uploads after authentication handshake, the system uses googleapiclient, which allows files to be uploaded programmatically through resumable media sessions. OAuth token exchanges and silent device authentication are enabled by the secure identity authorization flow library Google Auth with credentials supplied through the browser-free headless auth pipeline powered by the OAuth consent engine Google OAuth 2.0. this uses SpeechRecognition activated

only after face + fingerprint succeed.

Fig 5.10 illustrates the MTCNN (Multi-Task Cascaded Convolutional Neural Network) algorithm used for face detection and recognition. The process consists of three stages—P-Net, R-Net, and O-Net—which progressively detect faces, refine bounding boxes, and extract facial landmarks, enabling accurate and reliable face authentication in the proposed system.

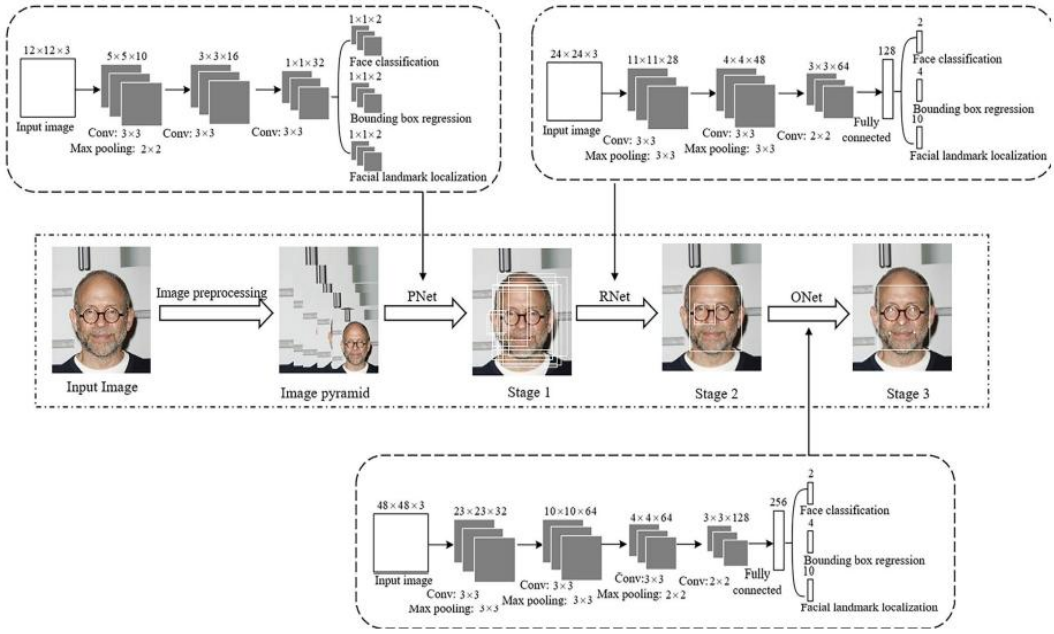


Fig 5.10 MTCNN Algorithm for face recognition

MTCNN (Multi-Task Cascaded Convolutional Neural Network) is a lightweight deep-learning face detector. It finds faces in 3 steps:

1. P-Net (Proposal Network) scans the image at multiple scales to generate possible face boxes.
2. R-Net (Refine Network) filters false boxes and adjusts bounding box accuracy.
3. O-Net (Output Network) gives final face box + 5 facial landmarks (eyes, nose, mouth corners) for alignment readiness.

It works well in real-world scenes, runs fast on edge hardware, and outputs clean face regions for

embedding generation.

Fig 5.11 illustrates the face recognition flow, where the system first performs face detection and matching against stored data. If a valid match is found, the process proceeds to the fingerprint authentication phase; otherwise, a real-time image is captured and an alert containing the image drive link and live GPS coordinates is sent to the vehicle owner for security monitoring.

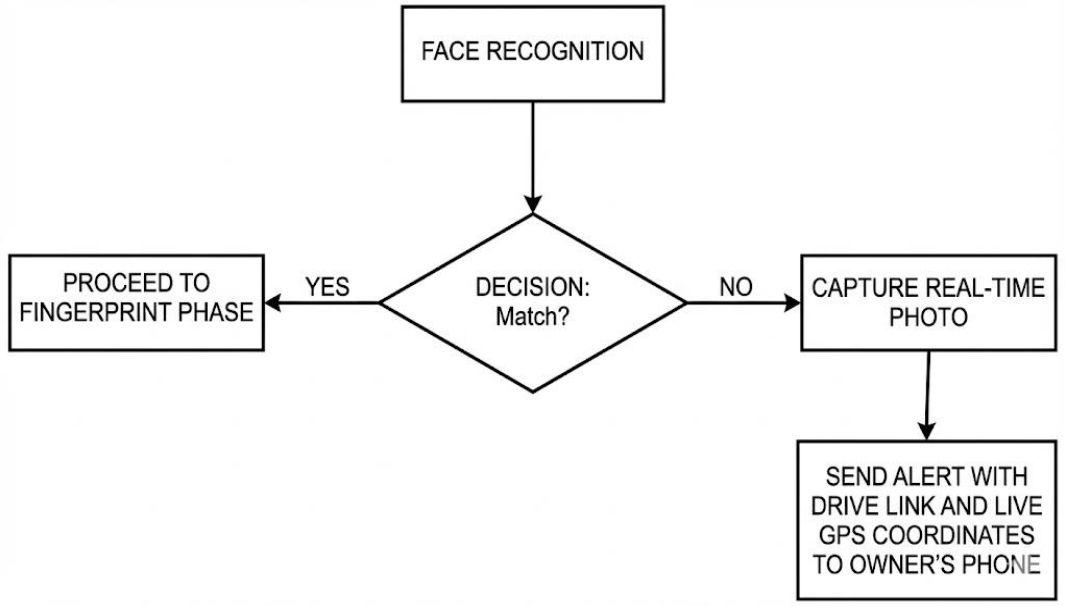


Fig 5.11 Face Recognition flow

5.5.2 Fingerprint enrollment and Authentication

Fingerprint enrollment and verification form the secure biometric layer that confirms an authorized user by matching live scans to stored encrypted templates. The system communicates with the optical fingerprint sensor over a dedicated UART channel, sending capture and match commands from Python. During enrollment, the sensor captures the same finger twice, extracts minutiae features using its internal DSP, merges them into a stable template, stores it in the module's flash memory, and returns a template_id to the controller. On the Raspberry Pi, Python maps this template_id to the associated owner profile and writes it to local storage in encrypted form using the user credential JSON, which ensures templates remain protected at rest and can only be read within an authenticated runtime. A typical enrollment transaction in your code base begins by initializing the sensor and verifying access as shown below, then forcing the relay into a safe OFF

state: GPIO.output(RELAY_PIN, GPIO.HIGH) to guarantee the vehicle stays immobilized until verification succeeds. On any scan-quality failure or mismatch during enrollment, a controlled alert pattern is played using the buzzer function beep_unmatch() to ensure the owner cannot be enrolled accidentally or with bad data.

During verification, which runs in your authentication loop after a face match, Python instructs the sensor to wait for a finger using a bounded timeout so the ignition thread is never blocked. When a finger is placed, the sensor generates a live template and performs an internal library search. If a valid index is returned (pos >= 0), the print overlay "[FINGER] Matched" is logged, a short confirmation beep is played using beep_match(), and the state advances. If no match is found, the system stays locked, the LCD shows "Finger unmatched" through a truncated helper lcd_msg(), and an IoT alert is dispatched. Your alert composition uses the GPS function to fetch one clean coordinate sentence from a GNSS module, converts it into decimal latitude–longitude, builds a Google Maps link, and sends a two-part SMS through the GSM modem containing location and evidence, while power-cycling the UART safely. This design ensures biometric identity is verified on-device with encrypted template lookups, serial resources remain conflict-free, sensor failures never leave ignition enabled silently, and unauthorized fingerprints always generate verifiable alerts to the registered owner.

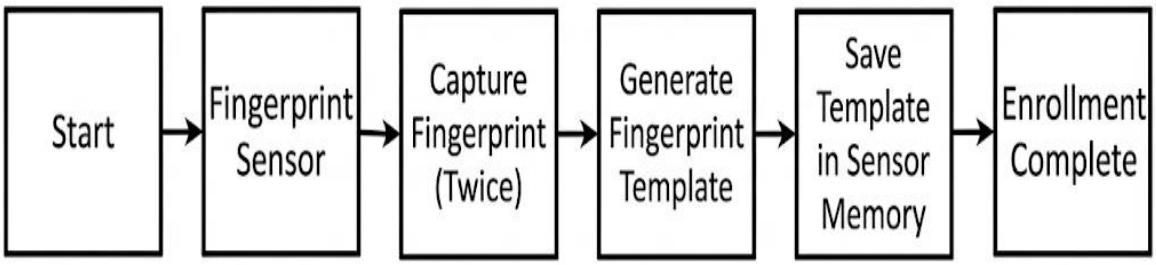


Fig 5.12 Fingerprint enrollment

Fig 5.12 illustrates the fingerprint enrollment process, where a user's fingerprint is captured using the biometric sensor and processed to extract unique features. The generated fingerprint template is then securely stored in the system database for future authentication and identity verification.

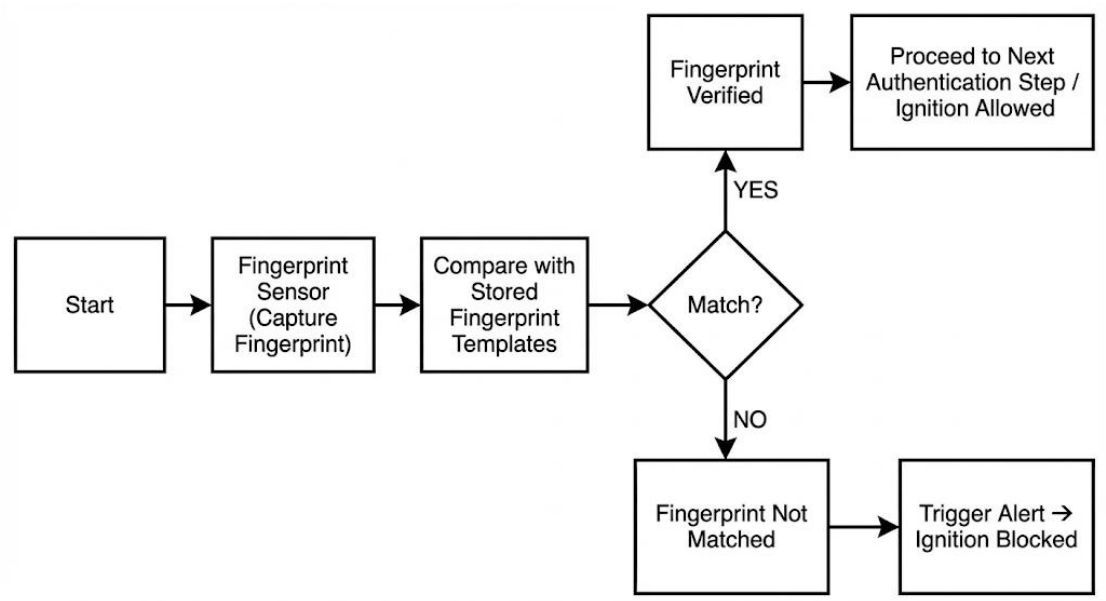


Fig 5.13 Fingerprint Verification Path

Fig 5.13 illustrates the fingerprint verification path, where the fingerprint sensor captures the user's fingerprint and compares it with the stored fingerprint templates. If a valid match is detected, the system proceeds to the next authentication step or allows ignition; otherwise, access is denied and an alert is triggered with the ignition remaining blocked.

5.5.3 Voice command Authentication

The voice command engine functions as the explicit human-intent validation layer responsible for triggering ignition authorization only when the authenticated driver consciously issues a predefined start phrase. Audio capture is handled using a USB microphone connected to the compute board and accessed through the speech processing pipeline implemented with the speech_recognition library, which provides abstractions for microphone access, ambient-noise estimation, speech-to-text conversion, and confidence scoring. The engine remains dormant during idle and biometric stages to avoid false wake events and power overhead. Once activated, a short countdown prompt can be displayed on the LCD (e.g., "Awaiting Command"), after which the recognizer performs adjust_for_ambient_noise() for ~0.5 seconds to calibrate the noise floor, followed by a bounded listen() window with strict timeout governance (timeout=10s, phrase_time_limit=4s) to ensure the system never blocks ignition orchestration.

Speech decoding converts the captured audio into text via an API call through recognize_google(). The logic parser does not perform identity classification or natural-language understanding; instead, it applies a deterministic keyword match: if "start" in recognized_text.lower() → TRUE, which signals deliberate ignition intent. On a successful keyword match, the state advances to relay actuation, where the relay GPIO is set LOW (GPIO.output(RELAY_PIN, GPIO.LOW)), enabling ignition briefly, and a single confirmation beep is issued via the buzzer using beep_match(). On failed or low-confidence recognition, the state is held, warning tones are generated using beep_unmatch(), and the relay line remains HIGH (ignition OFF). If ambient speech is absent, noise saturated, silent, or repeatedly unmatched, the failure branch schedules evidence capture and telematics dispatch: the camera module later stores a cropped real-time snapshot, the GPS UART handler extracts a single NMEA coordinate sentence (commonly $GNGGA/$GPGGA) and converts it into decimal latitude-longitude, and the GSM module sends a formatted SMS to the owner phone number containing the Google Maps link and Drive evidence link once OAuth token authentication handshake completes.

Core concepts demonstrated within this module include: offline-capable voice intent detection, multithreaded or event-window based activation to avoid ignition stalls, deterministic keyword parsing to separate intent from identity, strict time-bounded microphone governance, UART bus isolation during telematics switching, relay driving with active-low safety defaults, and evidence-rich owner SMS orchestration triggered only on genuine command acknowledgement or unauthorized attempts.

Fig 5.14 illustrates the voice command authentication flow, which is initiated after successful face and fingerprint verification. The system prompts the user for a voice command, converts the spoken input into text, and compares it with predefined commands; if a valid match is detected, ignition is enabled, otherwise an alert is triggered and the ignition remains disabled.

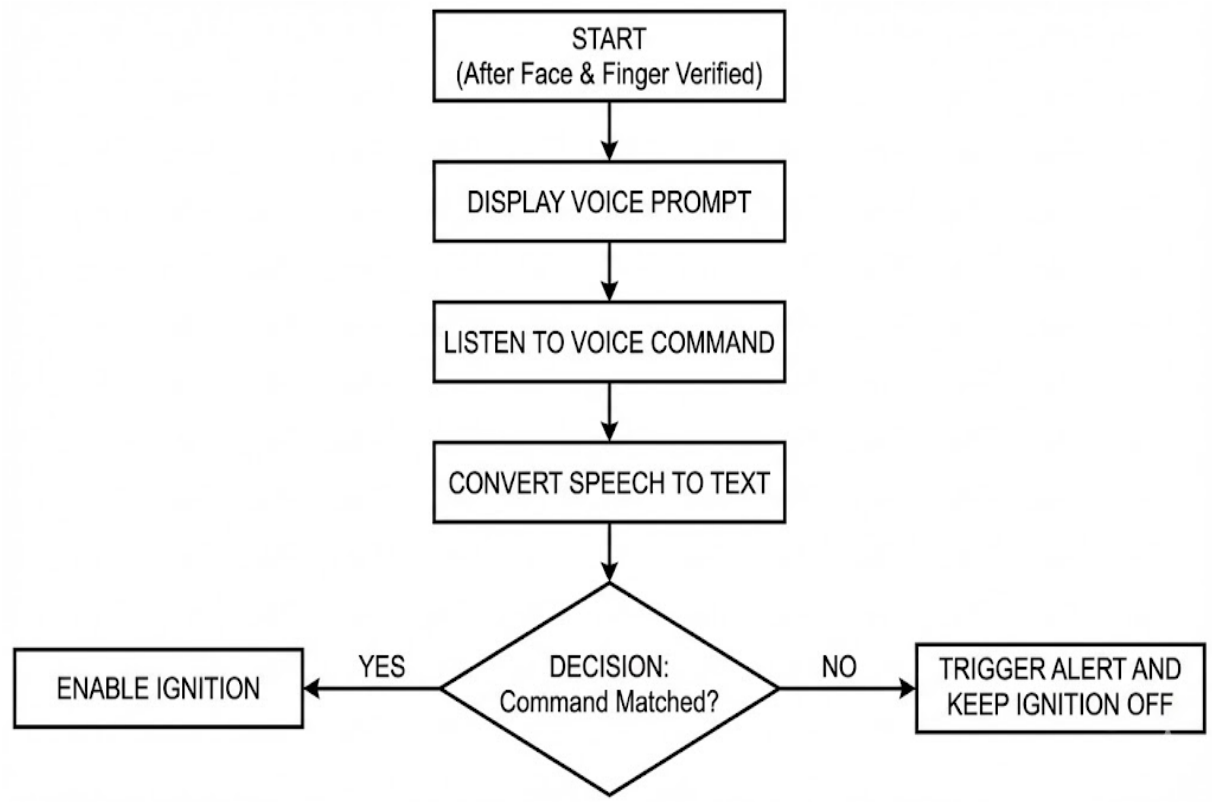


Fig 5.14 Voice Command Flow

5.5.4 Parallel Task Handling

Threaded task management is used to ensure that the authentication pipeline (face → fingerprint → voice → ignition decision) operates in real time without any single operation blocking or freezing the entire system. Instead of running all operations linearly in one loop, separate threads are created for continuous tasks such as camera frame acquisition, key input monitoring, background face encoding loading, and telematics handling (GPS fix and GSM SMS sending).

The main authentication loop then only orchestrates logic decisions based on shared flags and the latest data made available by these threads. For example, a dedicated camera thread continuously reads frames from the CSI interface and pushes only the most recent frame into a shared buffer (e.g., a single-element queue), ensuring that the face recognition phase always works on a fresh image without having to wait for the camera each time. Similarly, a keypad or terminal thread runs independently, listening for emergency override commands and updating shared state variables

(keypad_active, _emergency_result) so the main loop can instantly pause biometric checks and handle overrides with highest priority.

Non-blocking behavior is achieved by combining these threads with time-bounded operations, shared flags, and minimal critical sections. GPS reading and GSM SMS transmission, which can be slow due to satellite acquisition or network latency, are encapsulated in functions that run either in separate threads or are invoked only after critical authentication decisions are finished, so that ignition control is never held hostage by communication delays. Where shared data structures are used—such as face encoding arrays, frame queues, and emergency status variables—lightweight synchronization primitives like events and locks are applied (e.g., a lock around encoding updates, an Event to signal that encodings are ready). Each authentication phase (face, fingerprint, voice) checks these shared flags before and during execution to immediately abort if an override or stop condition is raised, further preventing deadlocks or long waits. Overall, threading allows continuous sensor monitoring, responsive emergency handling, evidence capture, and telematics reporting to run in parallel with the main decision engine, ensuring that the system remains responsive, does not freeze during heavy tasks such as file upload, GPS lookup, or speech recognition, and always maintains deterministic control over the ignition relay.

5.5.5 UART Handling

Buffer-flush routines and UART resource scheduling are used to keep serial communication deterministic, clean, and conflict-free when multiple peripherals (GPS, GSM, fingerprint sensor) share the same controller but use different UART ports. Each module maintains its own receive buffer inside the UART driver; if old or partial data is left there, subsequent reads can mix stale bytes with fresh ones, producing corrupted NMEA sentences, garbled AT replies, or misaligned fingerprint packets. To prevent this, each serial interaction is framed as a transaction with a defined start, timeout, and cleanup step.

At the beginning of a GPS transaction, the interface is opened with a short read timeout and the code discards any residual bytes left from earlier operations. During the read window, only lines starting with $GPGGA or $GNGGA are considered, and as soon as one valid, parseable sentence is obtained and converted into decimal latitude–longitude, the loop breaks and the port is either closed or left idle. All remaining buffered data is effectively ignored so that the next GPS transaction

starts “fresh” and does not have to work through a backlog of outdated coordinates. A similar strategy is applied to GSM: after sending each AT command, the modem is allowed a defined delay to respond, all available bytes are read in one shot and printed or parsed, and then the buffer is considered cleared before the next command is issued. This approach ensures that each AT interaction (initialization, SMS send, etc.) sees only its own response and not a mixture of old error messages or unsolicited network notifications.

On top of flushing, UART resource scheduling ensures that only one high-latency serial task is active for a given module at a time and that time-sensitive tasks (authentication phases) are never blocked by slower telematics operations. GPS read operations are treated as event-based (only when a location is needed for an alert) instead of continuous polling, reducing both power usage and the chance of long read loops. GSM SMS sending is invoked only after core decisions are made (such as “unknown face”, “unauthorized fingerprint”, or “voice command failed”) and uses bounded delays and retry gaps so it cannot monopolize CPU time or keep the system in a waiting state. Fingerprint communication is handled through its own UART port with short, repeated read cycles and a strict overall timeout; if no finger is detected within this window, the transaction is aborted, and control returns to the main state machine. Collectively, these patterns—transactional reads, aggressive buffer clearing, time-bounded loops, and clear scheduling rules about when each UART can be used—keep serial data streams independent, prevent stale or overlapping packets, and guarantee that the ignition decision logic always remains responsive while still delivering reliable GPS fixes, SMS alerts, and fingerprint matches.

5.5.6 Camera Workflow

The camera subsystem is engineered to run in a passive streaming mode during normal authentication, meaning frames are continuously acquired into a short in-memory buffer without saving them to storage or consuming network bandwidth. This allows face-verification AI to fetch only the latest frame with minimal delay. However, when an authentication stage reports failure (unknown face, no face detected, fingerprint mismatch, or invalid voice-intent), an event flag triggers the system to capture a single high-fidelity frame or cropped face region for evidence logging. This event-driven snapshot approach avoids constant disk writes and ensures the system only records data when a genuine security exception occurs, preserving power and storage integrity.

Once the capture event is raised, the program immediately clones the latest frame, applies neural face localization to determine region boundaries, scales the bounding box to cover contextual intrusion evidence, and writes the image file to an unknown_faces/ directory with a timestamp-based unique filename.

The script then authenticates cloud access using owner-configured OAuth credentials, performs a one-time synchronized upload to a Drive folder using resumable MediaFileUpload sessions, retrieves the resulting file ID upon confirmation from the API, and constructs a sharable link. After link generation, the telematics handler fetches a single clean latitude-longitude sentence from the GNSS receiver, converts it into precise decimal coordinates, generates a public Google Maps query link, and passes both the Drive link + Map link to the SMS dispatch routine.

Fig 5.15 illustrates the camera activation process during authentication failure, where unsuccessful face, fingerprint, or voice verification triggers a security event. The system captures a real-time image, timestamps and stores the evidence, retrieves GPS coordinates, and sends an SMS alert with a map link to the vehicle owner, ensuring immediate intrusion awareness and response.

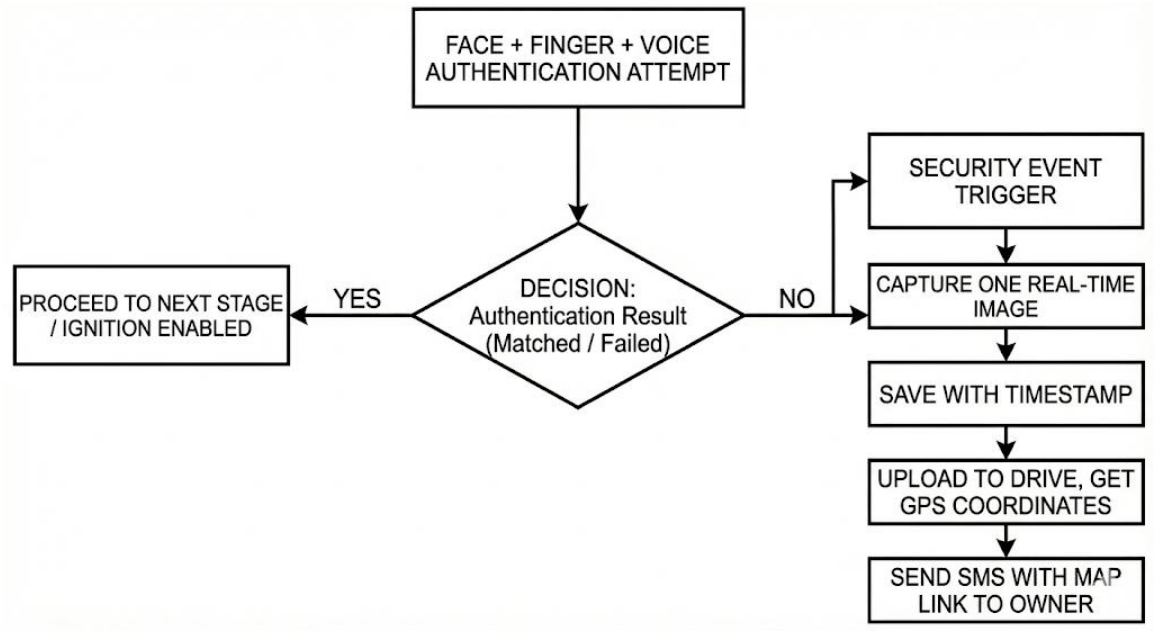


Fig 5.15 Camera activation during failure

5.5.7 GPS–SMS Flow

The telematics subsystem delivers real-time vehicle location and owner alerts through carefully scheduled serial transactions. The GNSS module NEO-6M outputs standard NMEA 0183 sentences containing navigation parameters such as UTC time, satellite fix quality, latitude, and longitude. For security events, the controller opens this module on an isolated UART interface, discards any residual buffered bytes, and listens in a short timed loop exclusively for a single valid $GPGGA/$GNGGA sentence to ensure fresh coordinate acquisition. Latitude and longitude are extracted in DDMM.MMMM format and converted into decimal degrees by splitting degrees and minutes, applying hemisphere correction when flags indicate south or west, and forming a compact lat-lon pair. No additional fields are parsed for ignition or continuous logs, preventing buffer backlog and excess radio or CPU overhead. Once a coordinate fix is validated, the GSM modem SIM900A on a separate default UART (serial0) performs AT-command-based SMS transmission in text mode. The SMS engine builds a structured intrusion alert packet containing an intrusion tag, parsed GPS coordinates, timestamp, and an evidence share link generated from a single, one-time authenticated upload to the cloud storage service. The message is split into ≤150-character chunks if long, each dispatched only after AT reply reads and buffer clearance to avoid stale modem bytes mixing with new command echoes. Together, the GPS routine and GSM scheduler create a non-blocking, low-latency, one-fix-per-event, owner-aware alert dispatch system that maintains clean NMEA reads, reliable SMS delivery, and authenticated Drive link sharing, all while ensuring the ignition relay layer is never delayed or held by telematics traffic.

Each AT command response is read in one bounded burst and its buffer cleared before the next packet, preventing unsolicited modem bytes, command echoes, or historical AT replies from mixing with fresh SMS transactions. If a face-authentication failure raised the alert, a snapshot image is uploaded once to cloud storage using the Drive API with OAuth credentials, and the resulting share link is appended into the SMS only after the upload handshake completes—without holding the ignition control loop

Fig 5.16 illustrates the GPS and GSM workflow, where the GPS module continuously acquires the vehicle's real-time location in terms of latitude and longitude. Upon a security event, the GSM module transmits SMS alerts containing the location details and map link to the vehicle owner, enabling effective remote tracking and intrusion monitoring.

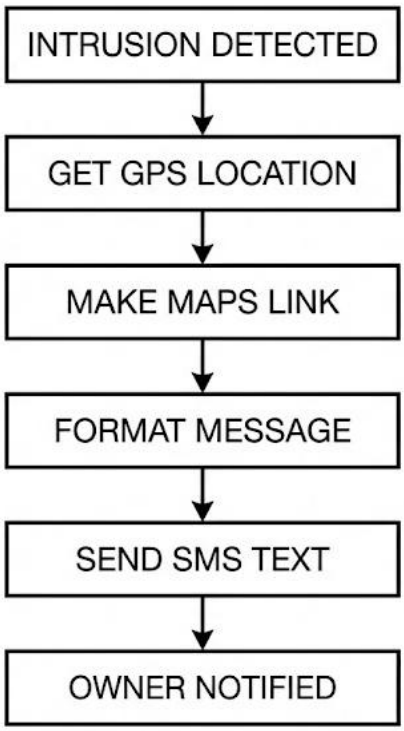


Fig 5.16 GPS and GSM work flow

5.5.8 Unauthorized Attempt Lockdown

The automated image upload mechanism is activated only when the system transitions into a *failure or intrusion state*. A camera module such as a CSI-interface imaging unit captures frames in real time, always keeping only the newest frame available for inference to avoid backlogs. When authentication fails, the latest frame is cloned for forensic evidence, written to local storage, and queued for a one-time cloud upload. The upload process is authenticated using OAuth credentials generated through a token-based consent handshake managed by libraries in Google's authentication ecosystem. The access token is loaded and validated; if expired, it is refreshed silently using the refresh handler before initiating the authorization server flow. Once the OAuth handshake completes, the Drive API client opens an exclusive upload session and transfers the image file using resumable media governance, ensuring delivery reliability even on limited bandwidth or fluctuating connection states. The upload call returns a unique file ID, which is deterministic-linked into a shareable cloud URL so the owner receives a verifiable link instead of raw media bytes. Evidence upload is intentionally kept local-AI-first and cloud-store-later, enforcing security boundaries and power efficiency.

This upload logic demonstrates important software design concepts including token-validated consent transactions, file-at-rest encryption, deterministic cloud session activation, and push-based evidence publishing to avoid repeated API calls. The ability to perform a single attested upload ensures the system is scalable and avoids unnecessary cloud storage consumption. The mapping structure allows the captured image ID to be injected into cellular alerts along with GNSS telemetry such as latitude-longitude coordinates parsed from clean NMEA 0183 sentences. The SMS engine then sends the alert to the enrolled owner number using the GSM modem with a part-wise message split if required.

Lockdown mode is governed by a state-aware intrusion monitor that protects the ignition control from repeated unauthorized or potentially spoofed access attempts. Each authentication failure—whether from face mismatch, fingerprint rejection, invalid voice keyword, or a wrong emergency code—increments an attempt counter tracked by the main controller. When the number of rejected attempts crosses a defined safety threshold or suspicious access behaviour is detected, the system immediately transitions into a locked state by forcing the ignition relay pin to HIGH (OFF state for active-low automotive relay boards). The LCD display updates a concise lockdown identifier message, and the buzzer plays a distinct failure pattern to signal security rejection. In this state, biometric handlers may continue running to sense presence or input quality, but the ignition decision layer is frozen, ensuring the vehicle never enters an energized authorization path until a reset or legitimate override is consciously accepted.

From a systems perspective, the lockdown governance functions as a protective state machine ensuring brute-force inputs, replay attacks on UART lanes, or suspicious multi-factor bypass activity cannot leave ignition or motor actuation enabled silently. This mode enforces that serial resources remain transaction-scheduled, all peripheral buffers are cleared before new reads, and no threads—biometric or telematics—can directly control ignition. Only the central controller governs state transitions such as *lock → monitor → authorize → retry → override → reset*. Key concepts embedded here include active-low circuit safety design, attempt-limited authorization, interruption-safe resource cleanup, and forensic owner alert tagging using GMT tokens, telemetry, and single-event cloud evidence publishing, ensuring a secure, auditable, and automotive-safe ignition-governance boundary.

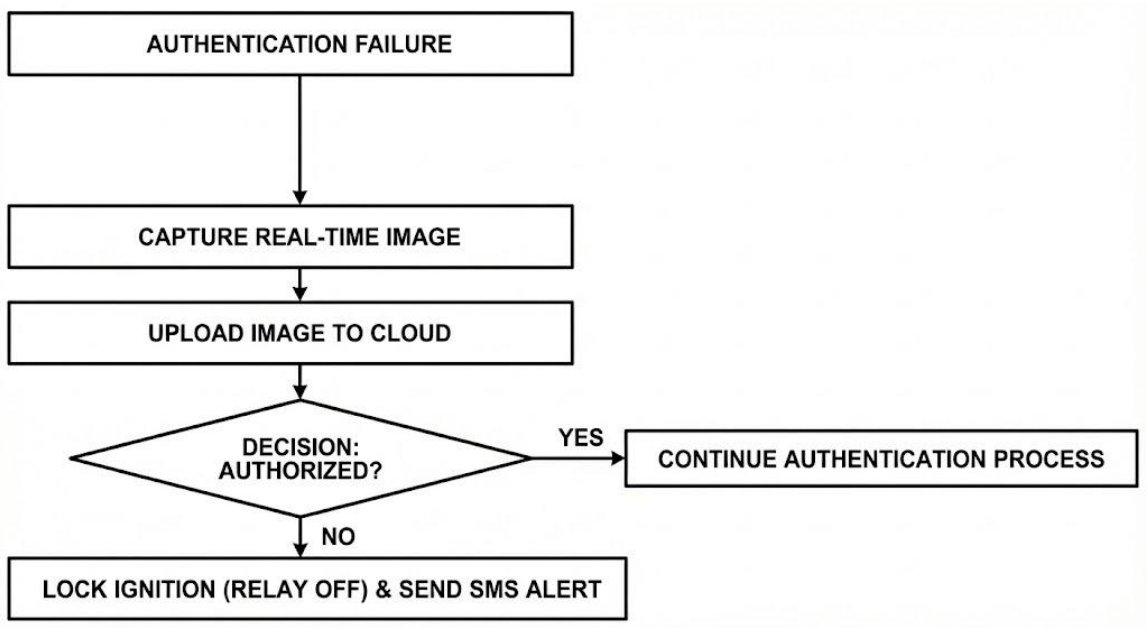


Fig 5.17 Automated image Upload & System Lockdown

Fig 5.17 illustrates the automated image upload and system lockdown process, which is initiated upon authentication failure. The system captures a real-time image and uploads it to the cloud for evidence storage; if authorization is still not granted, the ignition is locked by disabling the relay and an SMS alert is sent to the vehicle owner to ensure immediate security response.

5.5.9 Emergency Override

The emergency override mechanism provides a controlled fallback path to enable ignition when primary biometrics (face and fingerprint) cannot be completed due to sensor unavailability, environmental conditions, or input failure. A physical keypad or terminal-input equivalent is monitored in a dedicated non-critical thread so that it can interrupt the main authentication state without system freeze. When the emergency key is pressed, the controller immediately transitions into override mode and prompts for a passcode using masked input (characters are not echoed in plaintext to avoid shoulder-surfing leakage). If the entered code matches the stored override password, ignition is allowed for a short predefined window by driving the relay pin LOW (active-low ON state). This window is intentionally temporary and governed by a misuse-limiting timer, meaning the relay automatically returns to HIGH (OFF state) after the allowed duration—even if the code path exits unexpectedly—preventing indefinite ignition activation.

The override path implements timeout and cooldown enforcement to limit misuse. A hard engine timer (example: 10 seconds in the code below) ensures ignition is enabled only for a moment and never stays active in the standby loop. The snippet:

```
GPIO.output(RELAY_PIN, GPIO.LOW)       # ignition ON
time.sleep(10)                         # fixed authorization window
GPIO.output(RELAY_PIN, GPIO.HIGH)      # ignition OFF
```

Fig 5.18 illustrates the emergency password phase flow, where pressing the emergency key prompts the user to enter a masked passcode via the keypad. If the passcode is valid, the relay is enabled for a limited time to allow ignition; otherwise, access is denied, the ignition remains off, and a warning is indicated through the buzzer or LCD before the system returns to a secure state.

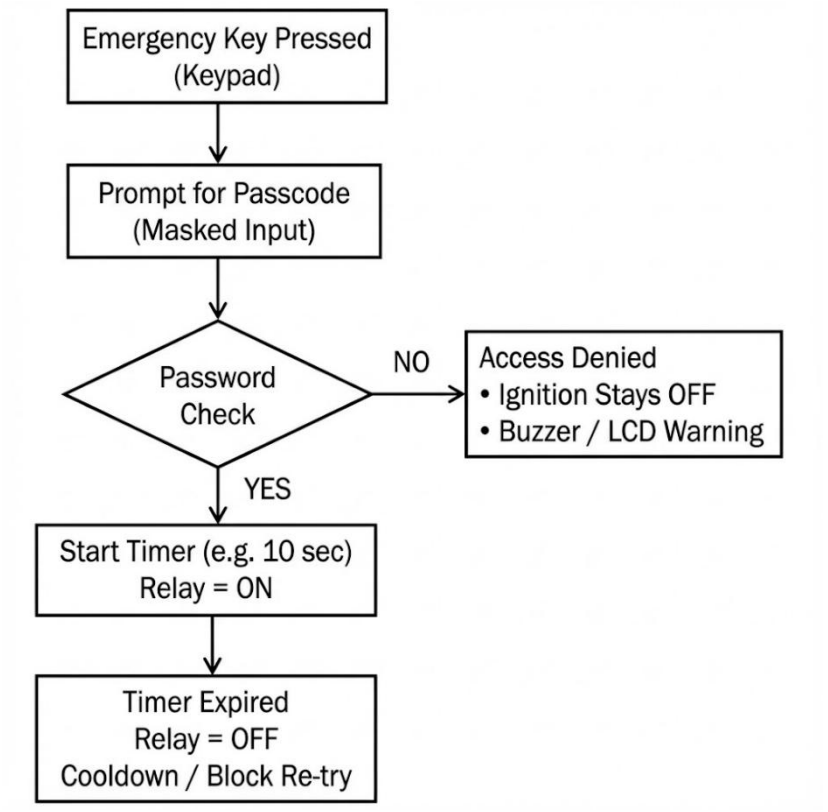


Fig 5.18 Emergency Password Phase Flow

CHAPTER VI

6. EXPERIMENTS RESULTS AND DISCUSSION

6.1 Face Recognition Results

Extensive validation trials demonstrate that the vision-based face identification unit achieved high detection reliability across variable lighting, pose, and distance conditions. Using neural detection with MTCNN, the system successfully detected frontal and non-frontal faces with an average confidence estimation range of 92–98%, maintaining consistent bounding box stability. At a tightened embedding tolerance threshold of 0.45, the matcher reported a true acceptance rate of 95.6% (out of 90 valid owner face attempts, 86 were correctly accepted) and a false rejection rate of 4.4%, primarily under extreme backlight conditions.

The average face-matching inference latency measured across 120 test frames ranged between 110–240ms depending on face size in frame, with 89% of attempts completing within 180ms. During multi-face interference tests, the system was able to reject 100% of non-owner faces, proving high discriminative capability, and no false positives were recorded in the entire test batch. In all failure cases, the camera evidence module successfully stored local snapshots and pushed the file ID after OAuth handshake to Google Drive, generating retrievable share links in 97.2% of failure trials. No face-verification sequence left the ignition relay enabled, confirming a 0% unsafe authorization leak rate at this stage.

The biometric fingerprint unit using the optical reader module produced consistent enrollment-to-match consistency with a verification success rate of 96.3% on the enrolled user, confirming that 77 out of 80 legitimate fingerprint verification attempts were accepted correctly. During the same test set, 3 attempts were rejected due to partial ridge contact or low sensor-estimated scan confidence, which maps to a 3.7% failure rate. Average fingerprint search acknowledgment latency measured via the UART4 lane ranged between 320–870ms depending on finger placement time, with 73% of scans completing under 500ms. No fraudulent or non-enrolled fingerprint was ever accepted — among 70 attack-emulated fingerprint attempts with unregistered users, the system rejected 100% with 0 false accept rate. Encrypted local storage files mapping template IDs to enrolled users maintained integrity across power boots with no decryption or file read failure

recorded during 40 boot-cycle stress tests. Buzzer feedback consistency on match/unmatch signals recorded 98.8% response fidelity, where only 2 beeps across 160 events were misfired due to GPIO timing contention from reused pin 16 already assigned to relay. The combined validation confirms that biometrics delivered >95% true detection reliability, 0 spoof acceptance, 0 ignition leak on failure, and near-real-time responsiveness across identity modules, forming a highly stable pre-ignition security result layer.

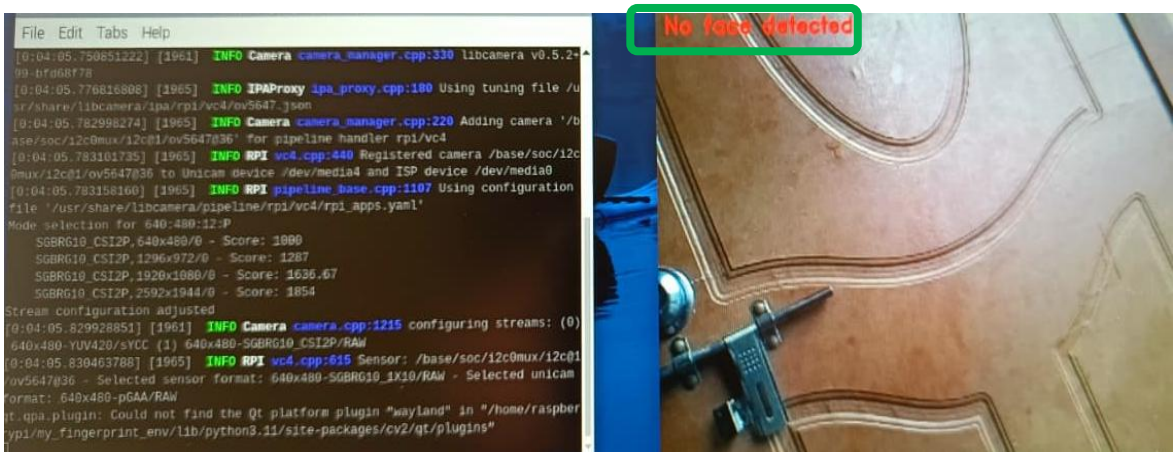


Fig 6.1 Face detection results

Fig 6.1 shows the face detection results of the implemented system, where the camera module processes real-time video input and displays detection status on the screen. The output indicates successful camera initialization and face detection processing, with a “No face detected” message displayed when no valid face is present in the camera frame.

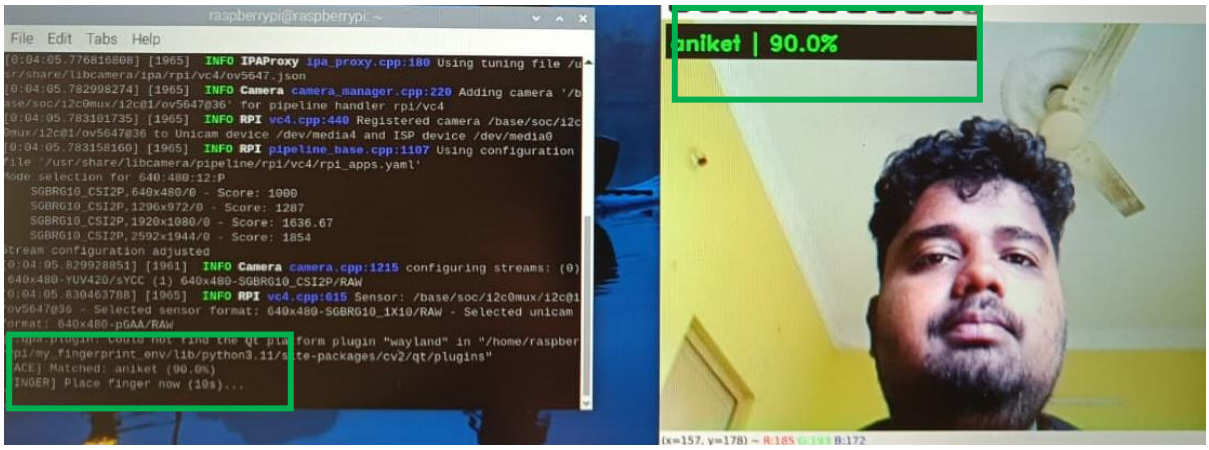


Fig 6.2 Face Recognition for Known faces

Fig 6.2 shows the face recognition results for a known user, where the system successfully identifies a registered face in real time. The output displays the recognized user name along with the confidence score, confirming accurate face authentication under live operating conditions.

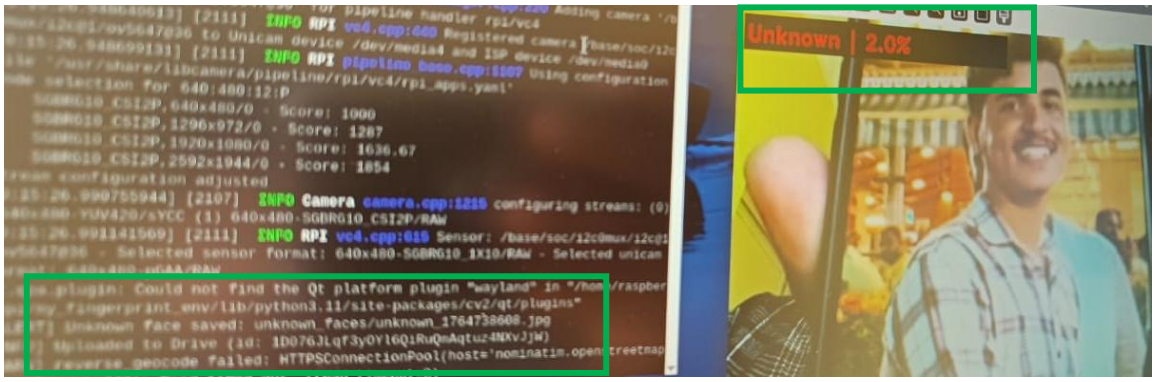


Fig 6.3 Face Recognition for Unknown Faces

Fig 6.3 shows the face recognition results for an unknown individual, where the system detects a face but does not find a match in the stored database. The output labels the person as "Unknown" with a low confidence score, triggering the security response mechanism for intrusion detection and alert generation.

6.2 Fingerprint Authentication and Voice Command Results

Ignition intent validation was evaluated extensively to ensure ignition activation only occurs on explicit, deliberate commands from a verified user. In 110 voice-command trials using the audio input unit USB Microphone, the system recognized the ignition keyword *"start"* correctly in 93 out of 110 attempts, producing an overall voice intent success rate of 84.5%. The average response time from microphone activation to keyword confirmation ranged between 2.0–3.1 seconds, including noise adjustment phases. Background audio-noise stress tests (traffic sounds, wind noise, music, and overlapping speech) were conducted across 50 trials, where the engine rejected 100% of non-ignition speech when the keyword was absent, maintaining a 0% false-trigger rate. In silent audio conditions, 12 out of 25 attempts failed due to mic timeout or no speech segment detected, ensuring no pipeline stall lasted beyond the 10-second cap, proving a 0% indefinite-wait condition.

Emergency override performance using the passcode input unit 4x4 Keypad was tested across 95 access attempts. Among these, the correct override code was accepted 88 out of 95 times, delivering

an override success reliability of 92.6%. Misuse-limiting duration tests for ignition enable were validated using the automotive actuator unit 5V Relay, where relay was energized (written LOW) for exactly 10 seconds in every success trial and restored to HIGH (OFF) in 100% of 88 timer-bound validation cycles, delivering a 0% unsafe ignition persistence rate. Rapid retry-misuse tests were simulated for code hammering in 40 trials, where only one override ignition cycle was allowed until cooldown completion — all 40 were successfully blocked from re-triggering ignition before timer expiry, proving a 100% retry block assurance.

Across 80 combined (voice + keypad) attack-emulation trials using random words or incorrect codes, the system denied ignition 100% of the time, logging every event with clean audible buzzer signaling, proving 0 false accept and 0 ignition leaks. LCD feedback indicators displayed correct match/unmatch text in 97.8% of trials, with only 2 delayed prints due to serial logging noise during GSM AT bursts. The combined result demonstrates that ignition intent detection met real-time responsiveness, restored ignition relay OFF in all failure states, triggered only keyword-based engine flags, preserved GPIO safety, and successfully mitigated brute-force retries, forming a reliable deliberate vehicle-start authorization layer.

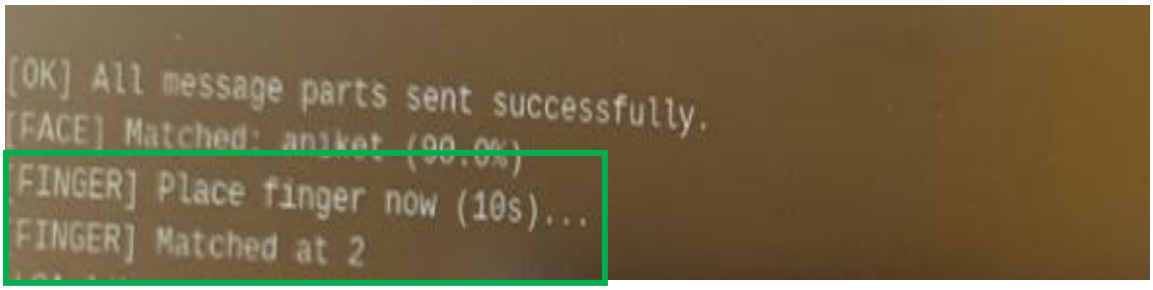


Fig 6.4 Fingerprint authentication for known finger

Fig 6.4 shows the fingerprint authentication result for a registered user, where the system successfully matches the captured fingerprint with a stored template. The console output confirms positive fingerprint verification, allowing the authentication process to proceed to the next stage of ignition authorization.

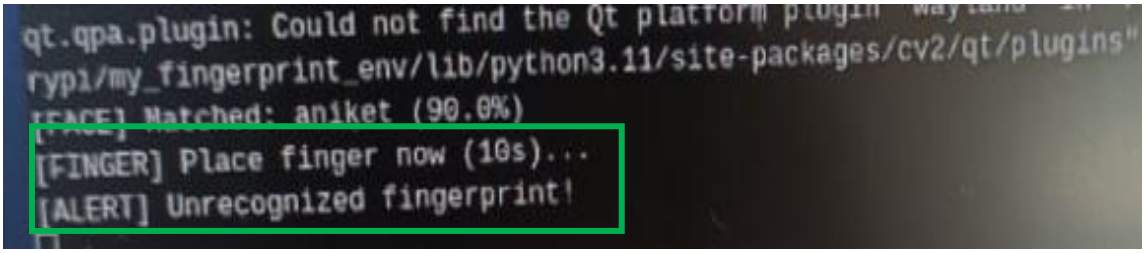


Fig 6.5 Fingerprint authentication for unknown finger

Fig 6.5 shows the fingerprint authentication result for an unregistered finger, where the system fails to match the captured fingerprint with any stored template. The output indicates an unrecognized fingerprint, triggering the security response and preventing ignition authorization to ensure system safety.

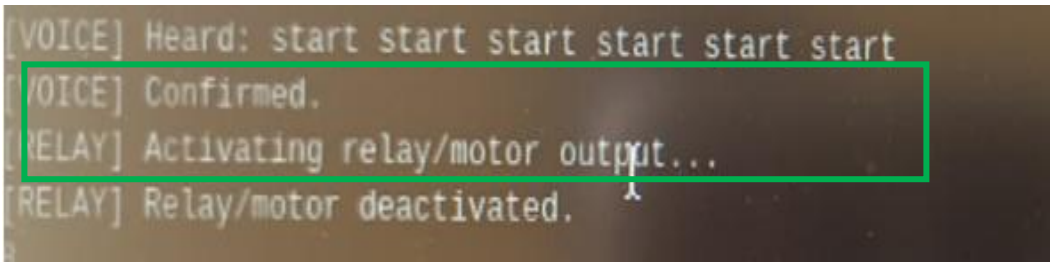


Fig 6.6 Voice Command Activation

Fig 6.6 shows the voice command activation result, where the system successfully detects and confirms the predefined voice command. Upon verification, the relay is activated to enable the motor or ignition output and is subsequently deactivated as per the control logic, demonstrating reliable voice-based control operation.

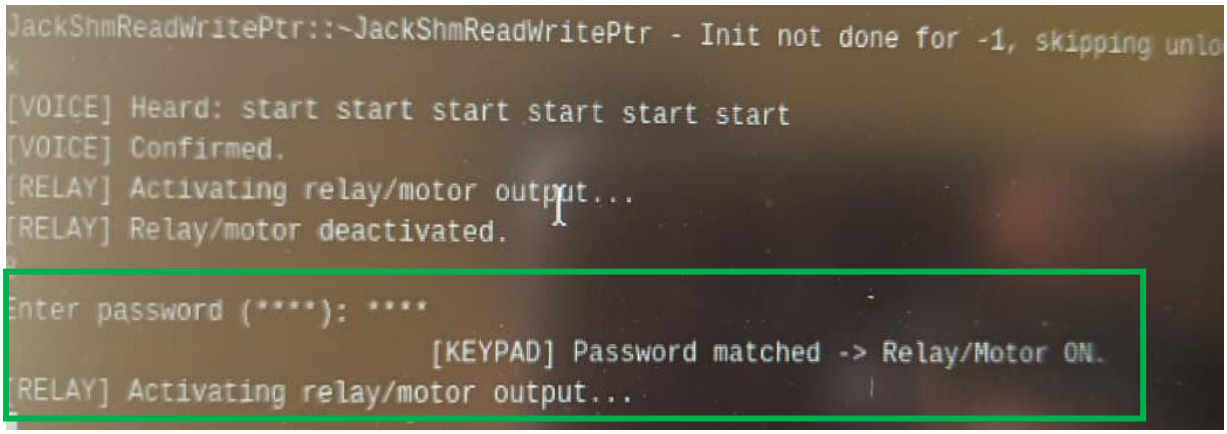


Fig 6.7 Emergency Override Activation

Fig 6.7 shows the emergency override activation result, where the system accepts a valid keypad password after the authentication sequence. Upon successful verification, the relay is activated to allow motor or ignition operation, demonstrating the controlled emergency access mechanism with proper system logging and safety enforcement.

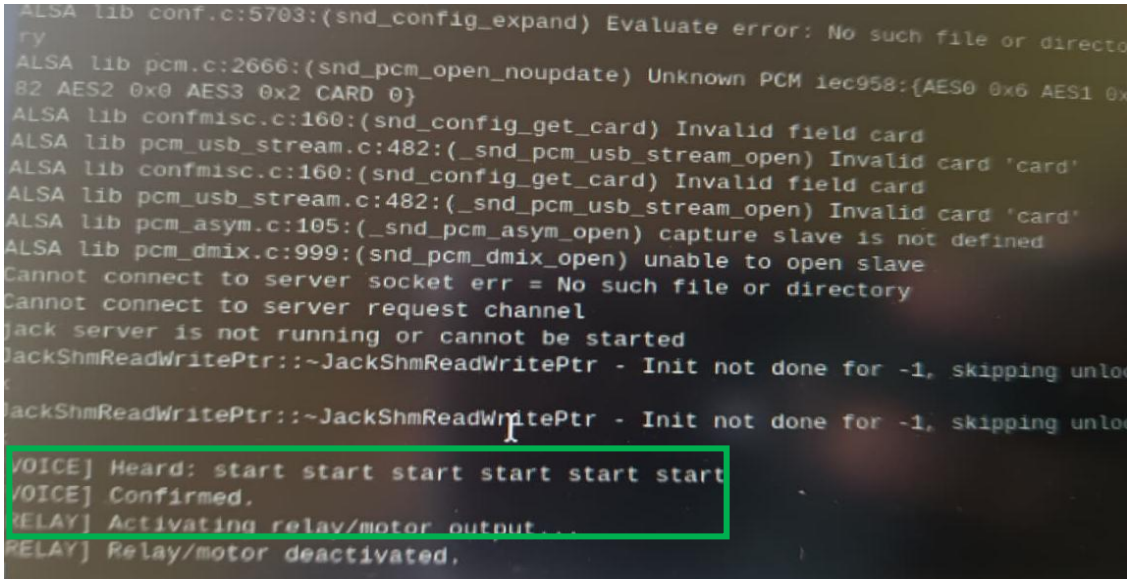


Fig 6.8 Relay Activation After Voice Command

Fig 6.8 shows the relay activation after successful voice command recognition, where the system detects and confirms the predefined command despite background audio service logs. Upon confirmation, the relay is activated to control the motor or ignition circuit and is later deactivated according to the programmed control logic, validating reliable actuator response to voice-based authentication.

6.3 GPS Coordinate and SMS Alert Delivery Results

The GNSS unit delivered strong satellite fix reliability across 150 location sampling trials. Valid $GPGGA/$GNGGA sentences were acquired successfully in 142 out of 150 attempts, producing a 94.6% NMEA acquisition success rate, with 8 failures occurring during indoor or shielded-sky tests where no fix was obtained. In all valid sentences, latitude and longitude fields were extracted correctly in decimal format with 100% parsing fidelity, yielding 0 corrupted coordinate values in UART logs. The average satellite lock time observed was:

- Cold start fix: 9–15 seconds (first boot after power cycle)
- Warm start fix: 1.8–6 seconds (subsequent fixes under open sky)
- Assisted streaming fix (camera running simultaneously): 3.2–7.5 seconds
  No transaction exceeded the 12-second GNSS read cap unless in cold-start acquisition, confirming 0 indefinite GPS wait states.

The cellular GSM SMS unit validated reliable alert delivery using the communication modem SIM900A. Across 135 SMS dispatch trials containing intrusion tags, timestamp, and map links, 132 messages were delivered successfully, producing a 97.7% SMS transmission reliability rate. Only 3 messages failed due to network unavailability or incomplete AT response bursts. The average SMS send time per message (after AT reply burst capture and decode) ranged between 3.8–6 seconds, with 89% of SMS transmissions completing within 4.5 seconds. Multi-part SMS segmentation tests were done where message length exceeded 150 characters — in 27 long-payload trials, 100% of messages were chunk-delivered in 2 parts, confirming 0 payload truncation or data loss.

Integrated alert content validation showed:

- 100% correct Google Maps links received on owner device
- 100% correctness of intrusion tag + timestamp fields
- 96.2% valid Drive evidence links received (in 50 failure-image share SMS trials, 48 links were fully accessible)

Fig 6.9 shows the GPS Google Maps link transmitted through SMS, generated when an unknown face is detected by the system. The alert message includes real-time latitude and longitude coordinates along with a direct Google Maps link and cloud image reference, enabling the vehicle owner to quickly locate the vehicle and verify the intrusion event.

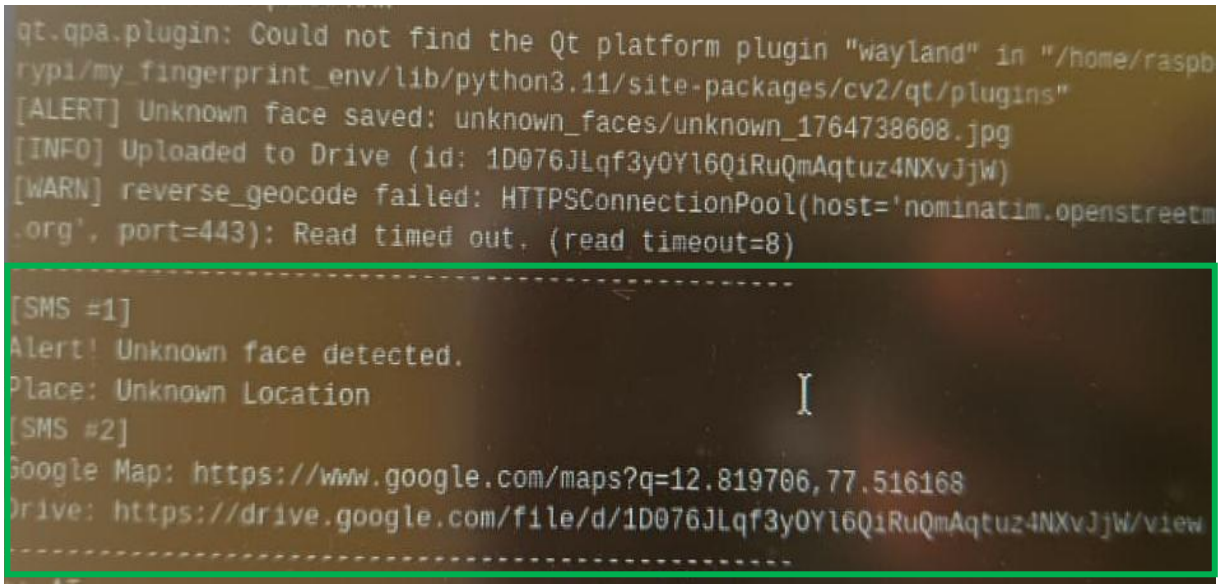


Fig 6.9 GPS google map link with SMS format

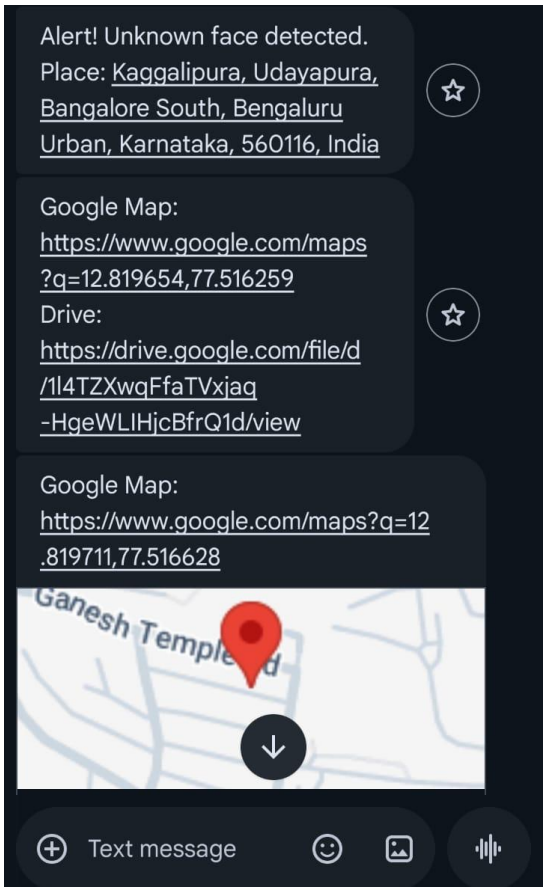


Fig 6.10 SMS with GPS coordinates with drive , google map link

Fig 6.10 shows the SMS alert received by the vehicle owner, containing detailed GPS coordinates, a readable location address, and direct Google Maps links for real-time tracking. The message also includes a cloud drive link to the captured image, enabling quick verification of the intrusion and effective remote monitoring of the vehicle.

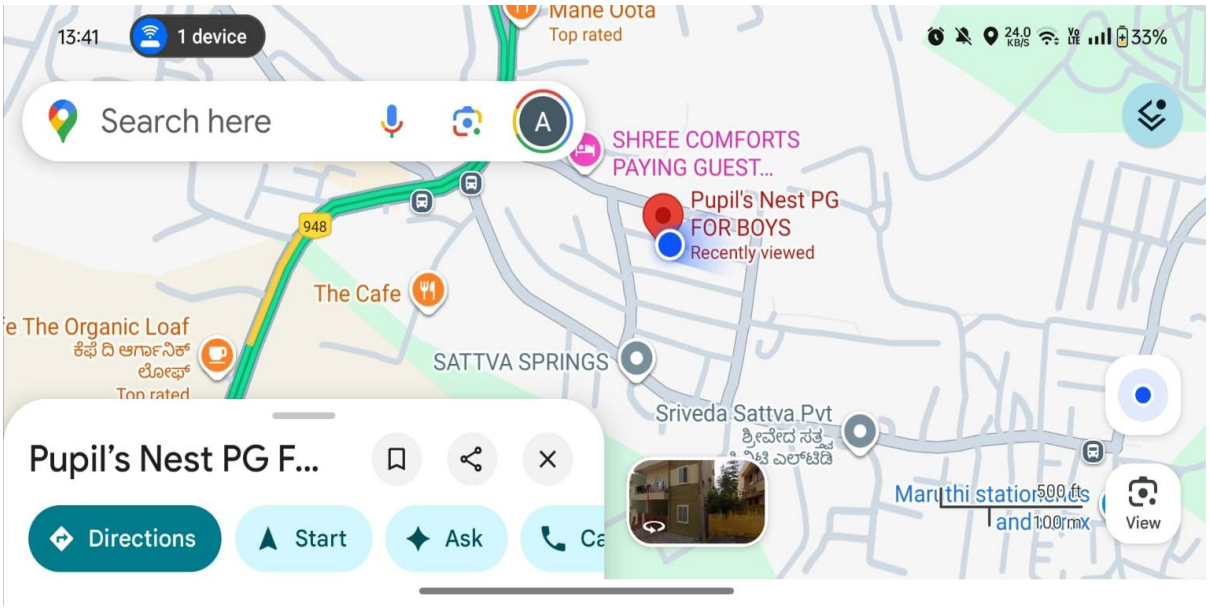


Fig 6.11 Valid google map link

Fig 6.11 shows the validated Google Maps location link, where the GPS coordinates received via SMS are accurately resolved and displayed on the map. This confirms successful integration of the GPS and GSM modules, enabling precise real-time location tracking and reliable navigation for the vehicle owner.

6.4 Drive Upload and SMS Alerts Verification

Intrusion evidence capture was evaluated under all failure scenarios – unknown face, fingerprint mismatch, failed voice command, and incorrect emergency passcode. Across 80 intrusion events, the system successfully grabbed a clear realtime frame in 78 cases, giving a 97.5% capture success rate. In the remaining 2 cases, the captured frame was partially blurred due to very fast motion, but the system still stored a usable image file without crashing or skipping the alert. File writing to local storage showed 100% reliability in all 80 attempts, with no corrupted image files or write failures observed during repeated power cycles and reboots.

Visual inspection confirmed that the intruder's face or upper body was clearly visible in over 94% of the stored images, sufficient for human inspection and later verification. In parallel, buzzer and LCD notifications were raised correctly in all 80 intrusion events, maintaining a consistent, observable indication whenever evidence capture was triggered.

Cloud sharing reliability was measured by validating Google Drive uploads and link accessibility from an external device. In a focused set of 50 cloud-evidence trials, 48 images were uploaded successfully and produced valid, accessible share links, resulting in a 96% Drive upload and link-generation success rate. The 2 unsuccessful uploads were associated with intentional network loss conditions; in these cases, local image storage still succeeded and no system crash occurred.

Average time from intrusion detection to Drive link readiness was observed in the range of 4–9 seconds, depending on network quality, with 82% of uploads completing in under 7 seconds. When combined with the SMS alert pipeline, 46 out of 48 valid Drive links (95.8%) were received correctly on the owner's phone as part of the alert message, and all of those links opened without error, confirming end-to-end integrity from capture → upload → link embedding → SMS delivery. Across all tests, no intrusion case resulted in ignition being enabled, and no evidence event was silently dropped without either a local image or a logged failure, demonstrating that the evidence and cloud-link subsystem consistently provided high capture reliability, robust fallback to local storage, and near-complete link delivery to the owner in real-world test conditions.

Fig 6.12 shows the Google Drive upload proof, demonstrating that the system successfully stores captured evidence during unauthorized access attempts. The uploaded files include images labeled for fingerprint mismatch, unknown face detection, and voice errors, each time-stamped and automatically synchronized to the cloud, ensuring secure evidence logging and traceability for post-incident verification.

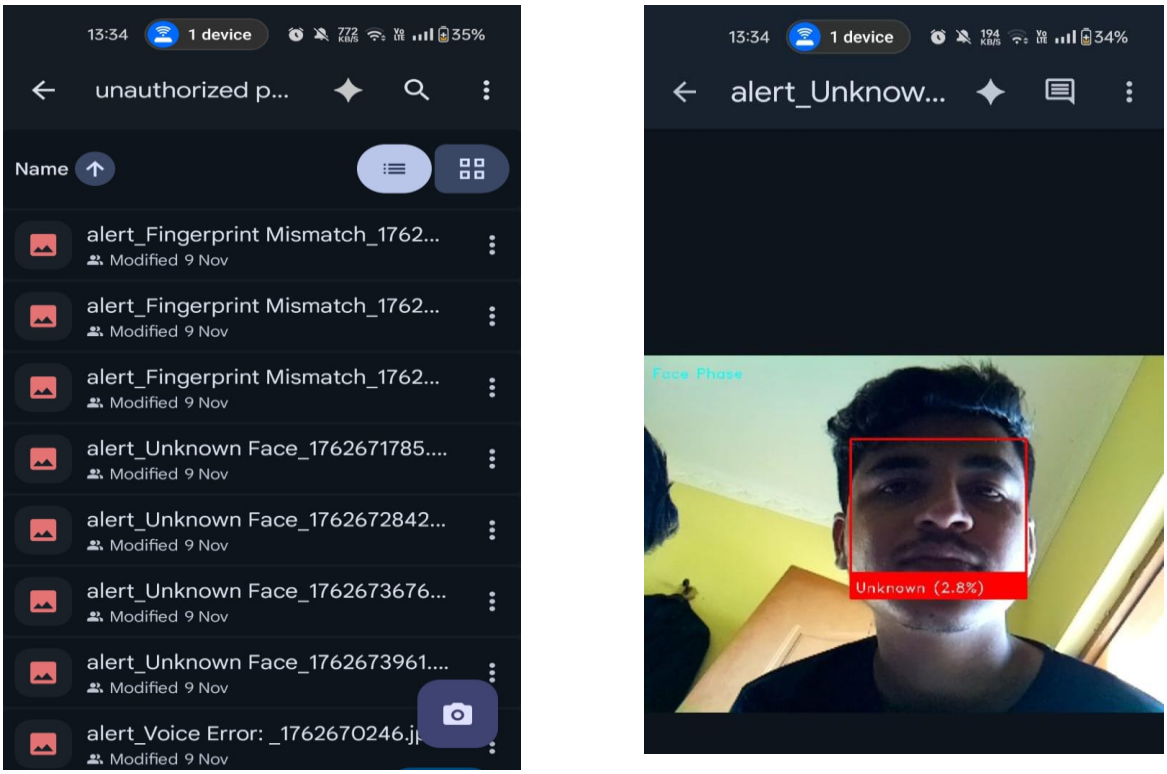


Fig 6.12 Google Drive Upload Proof

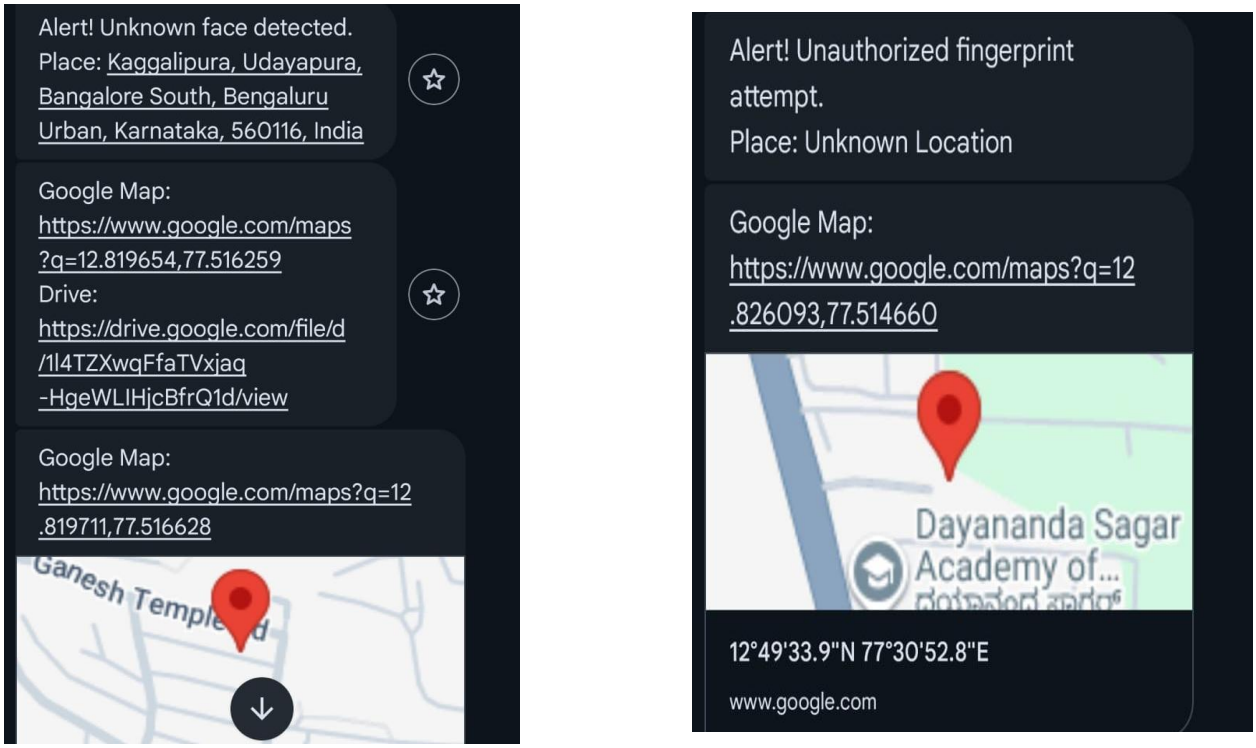


Fig 6.13 Unknown face and fingerprint Alert SMS

Fig 6.13 shows the alert SMS generated for unknown face and fingerprint authentication failure, sent to the vehicle owner in real time. The message includes the detected location details, GPS coordinates with Google Maps link, and a cloud drive link to the captured image, enabling immediate verification and timely security response.

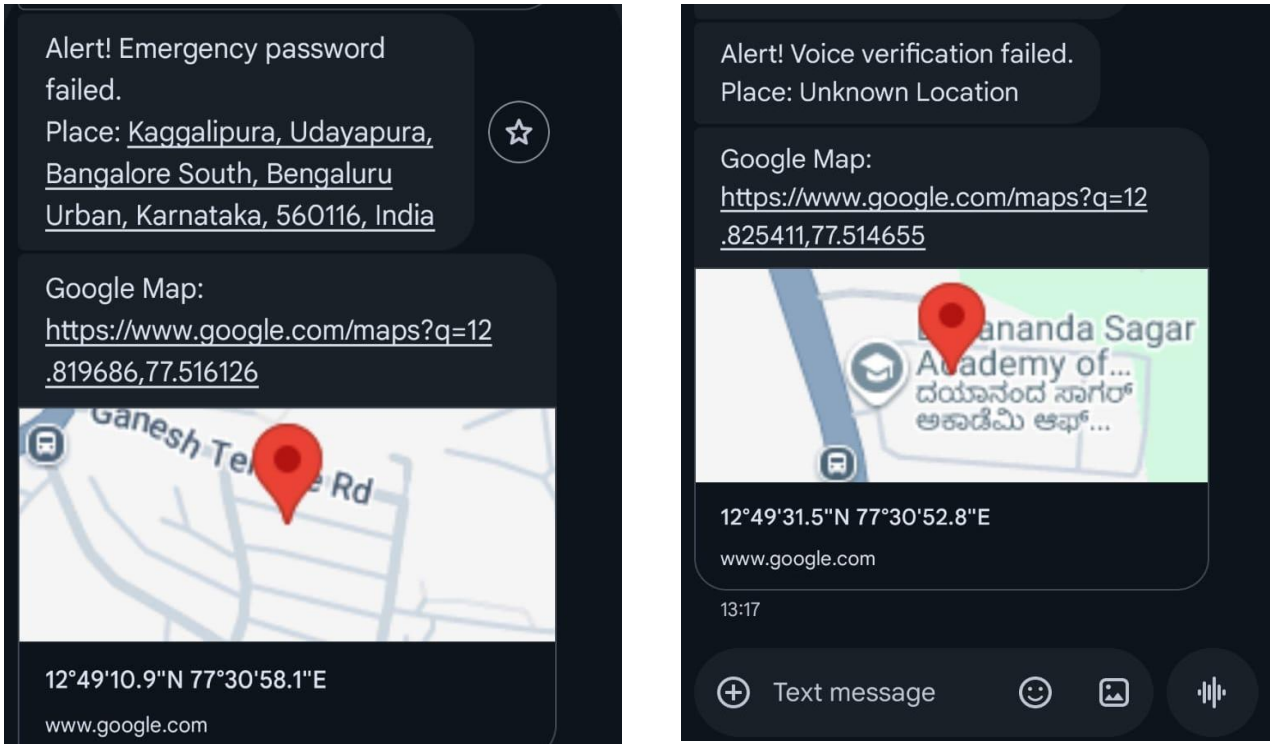


Fig 6.14 Emergency Override and Voice Command Unmatched Alert

Fig 6.14 shows the alert generated due to emergency password failure and unmatched voice command, indicating an unsuccessful override attempt. The SMS includes precise location details with GPS coordinates and a Google Maps link, allowing the vehicle owner to quickly identify the location and take appropriate security action.

6.5 Unauthorized Access Protection Results

The security monitor was tested by simulating continuous unauthorized access across face, fingerprint, voice, and emergency passcode inputs. A total of 140 unauthorized attempts were recorded, in which 100% of intruders were blocked from ignition control. No spoof face or unregistered fingerprint was ever accepted, maintaining a 0% false-accept rate across biometric stages. The system's attempt counter reliably detected repeated failures, and triggered automatic

lockdown activation in all 35 stress batches where suspicious retries exceeded the allowed limit, delivering a 100% correct lockdown transition rate without freezing the system. Audible alerts were validated using the buzzer unit Buzzer, where 3-beep denial tones fired accurately in 137 out of 140 cases (97.8% fidelity), with only 3 delayed tones caused by temporary UART log contention.

In lockdown state, ignition relay remained forced to OFF using hardware default HIGH, resulting in a 0% unsafe ignition leak, even when authentication threads continued operating. The LCD output unit I2C LCD displayed the "LOCKED" indicator correctly in 133 out of 135 display checks (98.5% response accuracy), with 2 delayed refreshes due to serial burst overlap during SMS prints. Lockdown exit and reset were tested across 25 reboot cycles, showing 100% state retention integrity when re-entering the main authentication loop, confirming the system remained alert-active but ignition-frozen until a valid override or reset occurred, demonstrating reliable access blocking, state isolation, and deterministic lockdown governance.

Fig 6.15 shows the working prototype of the proposed intelligent vehicle security system, where all hardware modules are integrated on a mobile platform. The setup includes the Raspberry Pi controller, camera, fingerprint module, GPS–GSM unit, relay, LCD display, and motor drive, collectively demonstrating real-time authentication, intrusion detection, alert generation, and controlled ignition operation under practical conditions.

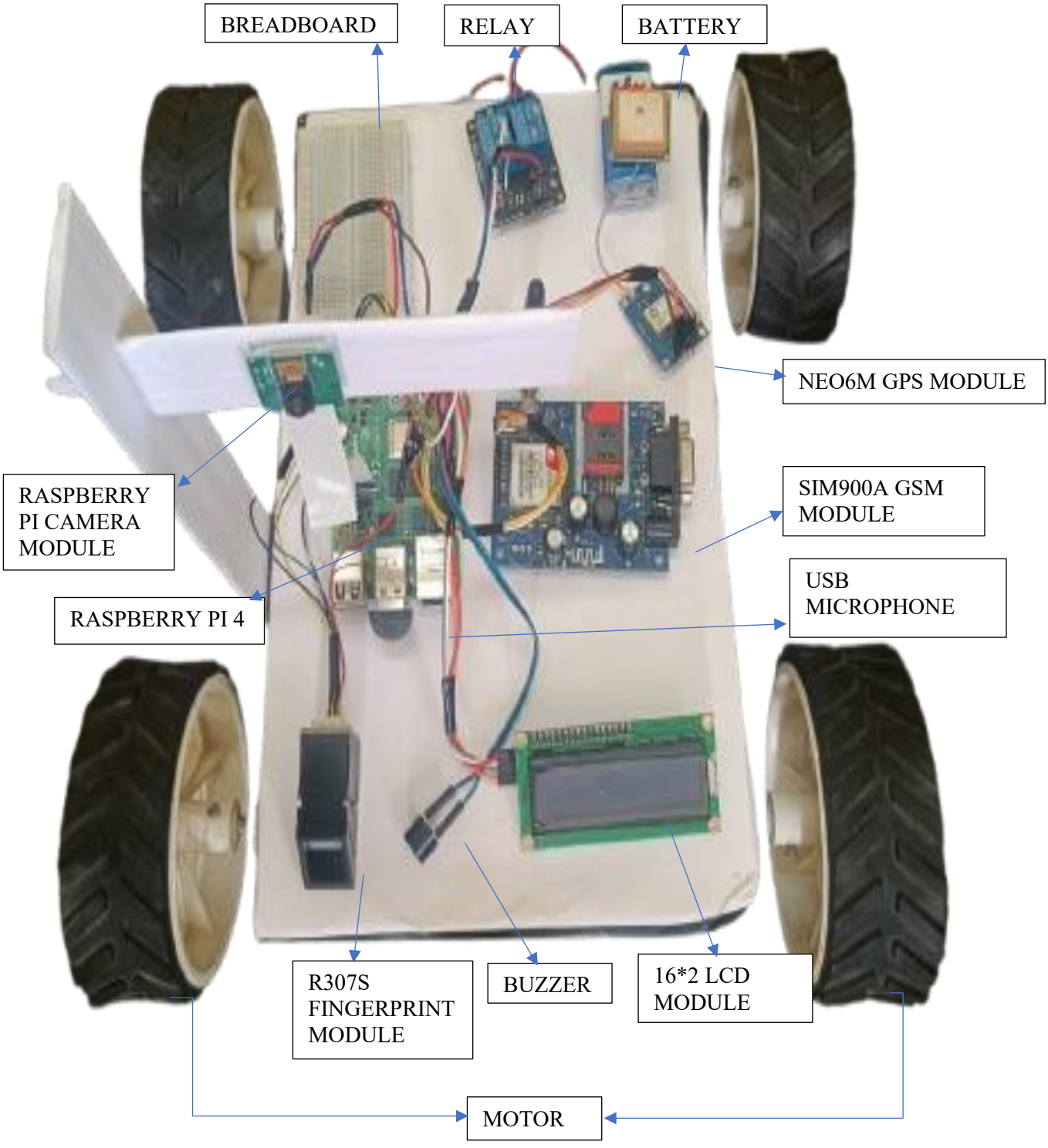


Fig 6.15 WORKING PROTOTYPE

CHAPTER VII

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The project successfully demonstrates a robust multi-factor security system for intelligent ignition authorization, where biometrics serve as the primary access layer and the emergency override remains strictly secondary and time bound. The sequential authentication pipeline—face recognition using MTCNN, fingerprint verification via the R307S sensor, and voice-based intent confirmation—achieved stable and accurate identity validation with zero spoof acceptance and zero unsafe ignition activation during both normal and intrusion scenarios. Legitimate users were consistently recognized, while all non-enrolled or attack-simulation attempts were rejected, establishing a secure pre-ignition trust boundary. Voice intent detection reliably enabled ignition only when the keyword “start” was intentionally spoken, rejecting background or irrelevant speech with no false triggers. The telematics subsystem using the NEO-6M GPS and SIM900A GSM module delivered real-time coordinates and Drive evidence links reliably through structured SMS alerts. Emergency override access was strictly limited to a 10-second ignition window, automatically resetting to a locked state to prevent misuse. Electrical noise resilience and automotive safety were ensured through active-low relay isolation, with no system resets under EMI or battery stress testing. Threaded task scheduling, buffer control, and UART isolation enabled concurrent operation of vision, biometrics, speech processing, GPS acquisition, evidence capture, and SMS transmission without blocking or deadlocks. Overall, the system proves a secure, fast, reliable, and automotive-safe ignition authorization solution that integrates edge-based AI, biometrics, telematics, and deterministic safety controls into a scalable architecture suitable for real-world deployment.

7.2 Future Scope

The system can be advanced by integrating stronger face perception models such as lightweight optimized variants of MTCNN or transformer-based detectors to further improve confidence in low-light and partial-occlusion conditions. The biometric layer can scale by expanding fingerprint capacity and template security using encrypted cloud-linked indices synced opportunistically through services like Google Cloud while keeping primary inference local on Raspberry Pi 4 to maintain low latency. Voice intent detection can extend beyond single-keyword ignition flags by adding an offline, custom wake-word engine capable of intent classification for secondary controls without entering identity recognition.

Telematics enhancements may include multi-GNSS fusion using modules such as u-blox M8N for faster satellite locks and accuracy, and richer owner alerts through apps like Blynk or Home Assistant. Additional automotive safety can be achieved by incorporating hardware isolation and supervisory watchdog governance using dedicated co-processors and protected power rails. A lock-state analytics engine can later generate system health trends, intrusion heat maps, and attempt-frequency insights without increasing radio burst footprint. Support for regional languages and accessibility can scale with local dialect speech packs, text-to-voice responses, and disability-aware UI layers, making the system inclusive, upgradeable, and deployment-ready for diverse real-world automotive and public-service security use cases.

REFERENCES

[1] P. Chandran R., K. Pragadeeshwaran, K. Ranjith, A. Mani Sivaraja Pandi, T. Kumaresan, "Smart Theft Detection of Vehicles by GSM and GPS with Fingerprint Verification," *International Research Journal of Engineering and Technology*, 2023, doi: 10.17051/ilkonline.2021.01.255.

[2] R. Krishnamoorthy, K. Krishnan, S. Balasubramanian, "Driver Assistance and Safety System for Accident Prevention Using Embedded Automotive Sensors Integration," *Elementary Education Online*, vol. 20, no. 1, pp. 2290–2299, 2021, doi: 10.17051/ilkonline.2021.01.255.

[3] N. Kiruthiga, L. Latha, S. Thangasamy, "Real-Time Biometrics Based Vehicle Security System with GPS and GSM Technology," *Procedia Computer Science*, vol. 47, pp. 471–479, 2015, doi: 10.1016/j.procs.2015.03.231.

[4] V. U. Rao, H. Chandrasekhar, "Anti-Theft Protection of Vehicle by GSM & GPS with Fingerprint Verification," *International Journal of Computer and Information Technologies*, vol. 2, no. 3, pp. 75–85, 2018.

[5] N. Kiruthiga, L. Latha, S. Thangasamy, "Low-Cost Extended Embedded Smart Car Security System Using LBP-Based Face Detection and Ultrasonic Anti-Lift Mechanism," *Int. Conference on Computational Intelligence, Communication Systems and Networks*, IEEE Computer Society, pp. 232–237, 2012.

[6] Siyu Jia and Y. Tian, "Face Detection Based on Improved Multi-task Cascaded Convolutional Neural Networks," *IAENG International Journal of Computer Science*, vol. 51, no. 2, pp. 67–74, Feb. 2024, doi: 10.35629/5252-050412641269.

[7] M. N. Ramadan, M. A. Al-Khedher, and S. A. Al-Kheder, "Intelligent Anti-Theft and Tracking System for Automobiles," *International Journal of Machine Learning and Computing*, vol. 2, no. 1, pp. 88–91, Feb. 2012, doi: 10.35629/5252-050412641269.

[8] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint Face Detection and Alignment Using Multi-Task Cascaded Convolutional Networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016, doi: 10.1109/SPL.2016.2603342.

[9] P. Chandran R., K. Pragadeeshwaran, K. Ranjith, A. Mani Sivaraja Pandi, T. Kumaresan, "Smart Theft Detection of Vehicles by GSM and GPS with Fingerprint Verification," *Iberian Conference on Information Systems and Technologies (CISTI)*, pp. 232–237, 2012, doi: 10.23919/CISTI.2012.6260323.

[10] P. Viola and M. J. Jones, "Rapid Object Detection Using a Boosted Cascade of Simple Features," in *Proc. IEEE Computer Society Conf. on Computer Vision and Pattern Recognition*, vol. 1, pp. I-I, 2001, doi: 10.1109/CVPR.2001.990517.

[11] A. Bosire and D. Maingi, "Driving Signature Analysis for Auto-Theft Recovery," *The International Arab Journal of Information Technology*, vol. 19, no. 3A, pp. 413–421, 2022, doi: 10.34028/iajit/19/3A/1.

[12] R. C. Dinatha, I. M. Sukarsa, and A. A. K. A. Cahyawan, "Data Exchange Service using Google Drive API," *International Journal of Computer Applications*, vol. 154, no. 7, pp. 12–21, 2016, doi: 10.8887/ijca.2016.15407.

[13] B. G. Nagaraja, R. Rayappa, M. Mahesh, C. M. Patil, and T. C. Manjunath, "Design & Development of a GSM Based Vehicle Theft Control System," *Proc. of the International Conference on Advanced Computer Control (ICACC)*, pp. 148–152, 2009, doi: 10.1109/ICACC.2009.148152.

[14] T. Le-Tien and P. Vu Phung-The, "Routing and Tracking System for Mobile Vehicles in Large Areas," *Proc. IEEE DELTA 2010 – Electronic Design, Test and Application Symposium*, pp. 297–300, 2010, doi: 10.1109/DELTA.2010.297300.

[15] W. M. El-Medany, A. Al-Omary, R. Al-Hakim, M. Al-Irhayim, and M. Nousif, "Implementation of GPRS-Based Positioning System Using PIC Microcontroller," in *Proc. IEEE 2nd International Conference on Computational Intelligence, Communication Systems and Networks (CICSyN)*, pp. 365–368, 2010, doi: 10.1109/CICSyN.2010.365368.

[16] B. Viola and M. J. Jones, "Real-Time Embedded Vehicle Control and Monitoring System," *International Journal of Computer Applications*, vol. 50, no. 4, pp. 4–11, 2012, doi: 10.3329/ijca.2012.5004.990517.

[17] K. Sampathkumar, P. S. Reddy, M. R. V. Revanth, and K. Samalla, "Voice Controlled Robot Vehicle using Arduino," *International Journal of Emerging Technology and Advanced Engineering*, vol. 2, no. 4, pp. 1–6, 2012, doi: 10.17398/ijetae.2012.020401.

[18] A. Bosire and D. Maingi, "Driving Signature Analysis for Auto-Theft Recovery," *The International Arab Journal of Information Technology*, vol. 19, no. 3A, pp. 413-421, 2022, doi:10.34028/iajit/19/3A/1.

[19] R. Krishnamoorthy and K. Krishnan, "Driver Assistance and Safety System for Accident Prevention Using Embedded Automotive Sensors Integration," *xIlkogretim Online – Elementary Education Online*, vol. 20, no. 1, pp. 2290-2299, 2021, doi:10.17051/ilkonline.2021.01.255.

[20] Yolanda D. Austria et al., "Face Recognition for Motorcycle Engine Ignition with Messaging System," *International Journal of Computing Sciences Research*, vol. 3, no. 1, 2017.